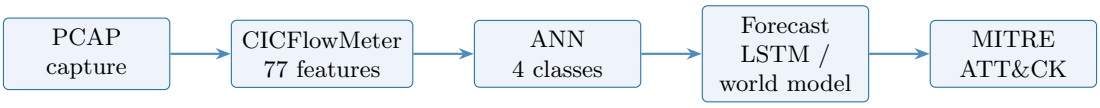
NIDS

System Field Guide & AI Roadmap

A plain-English tour of every part of the dashboard,
the attacks it can and cannot see, and where it should go next



Generated August 30, 2026

Offline system — no cloud, no telemetry, no external call at inference time

Contents

How to read this document

This guide has six parts.

- **Part I** explains what the system does *today*, one stage at a time, in language a non-technical reader can follow. Each stage has a picture of the real dashboard and a small flow diagram.
- **Part II** is a reference table of every function in the code, one line each, so an engineer can find the exact place a thing happens.
- **Part III** lists the network attacks that exist and shows, for each, how this app can *flag* it, *predict* it, and *prevent* it — and honestly marks what it cannot do yet.
- **Part IV** looks at real attacks that hit Indian organisations and what was done about them.
- **Part V** surveys the 2026 open-source and AI landscape — the projects that have already solved large parts of this problem.
- **Part VI** is the recommended plan: adopt the solved pieces, keep our forecasting engine as the thing that is genuinely new.

A one-line summary of the system: *it watches network traffic, decides whether the traffic looks like an attack, predicts what the next few seconds look like, and explains its reasoning against the MITRE ATT&CK catalogue — all on one machine, with nothing sent anywhere.*

Part I — What NIDS does today

1 The big picture

Network defence usually labels each "flow" (one conversation between two computers) as good or bad and stops there. That throws away *time* — the order in which an attacker probes ports, the way a burst of half-open connections comes before a flood, the reconnaissance that happens before lateral movement. NIDS keeps the time dimension: it rolls flows up into 10-second windows, learns how the "state" of the network moves from one window to the next, and forecasts the next few windows.

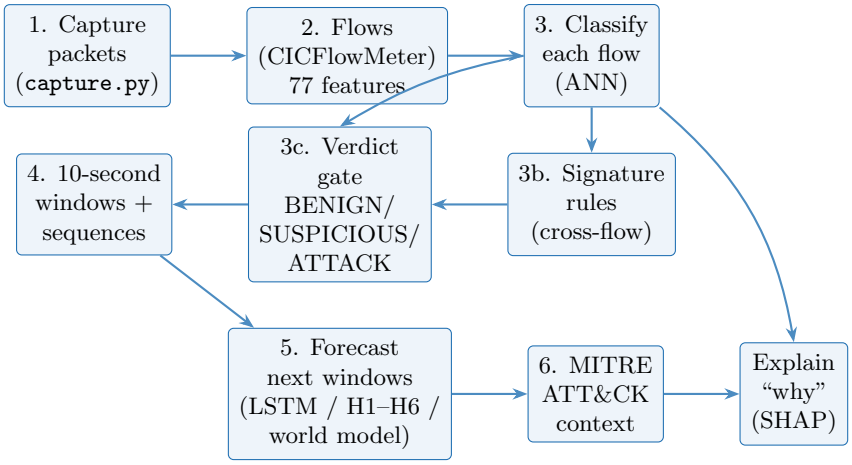


Figure 1: The whole pipeline. Everything runs locally; each arrow is a step a person or a timer triggers.

The pipeline in one paragraph. Raw packets are recorded to a `.pcap` file. CICFlowMeter turns that file into a table of flows, one row per conversation, each row summarised by 77 numbers (packet counts, byte totals, timing statistics, TCP-flag counts). A small neural network (the ANN) scores every flow as `BENIGN`, `DDoS`, `DoS` or `PortScan`. A separate rule layer looks across *all* flows at once to catch volumetric floods the per-flow view dilutes. A "verdict gate" decides whether the whole capture reads green (BENIGN), amber (SUSPICIOUS) or red (ATTACK). The labelled flows are then rolled into 10-second windows; a forecasting model predicts the next window's state (and a longer model predicts the next six). Finally a deterministic mapper attaches MITRE ATT&CK technique candidates and an operator playbook, and a feature-attribution step explains which inputs drove the call.

Fact	Value
Attack classes	BENIGN, DDoS, DoS, PortScan (4-way softmax)
Flow features	77 (CICFlowMeter / CICIDS2017 schema)
Window length	10 seconds (real timestamps, temporal pipeline)
Sequence length	5 windows in → 1 window out
Forecast horizons	1 window; also direct H1–H6; world model K=6 (max 24)
Verdict gate	a class reads red only if its own share of flows ≥ 20%
Deployment	single machine, offline, FastAPI + static dashboard on `:8765`

Table 1: The numbers that matter most, collected here and in Appendix B.

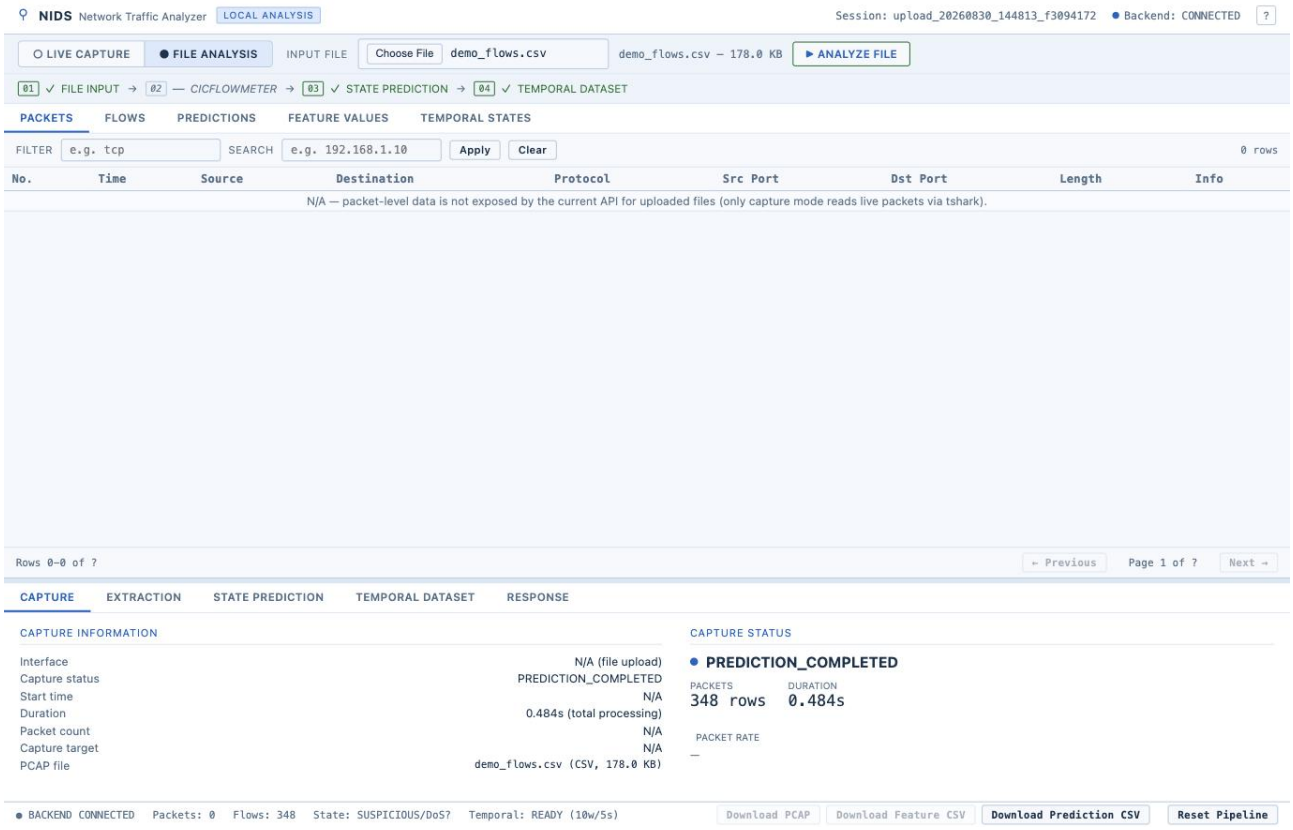


Figure 2: The dashboard after analysing a capture. Top strip: the four pipeline stages. Left: a table of packets / flows / predictions. Right: detail panels. Bottom bar: live counters and the current verdict.

2 Stage 1 — Capturing packets

What it does, in plain words

- Records live network traffic to a `.pcap` file, the standard format every analysis tool understands.
- Uses the operating system's own capture tool — `dumpcap` on Windows, `tcpdump` on macOS/Linux — started as a child process.
- Never invents numbers. The packet count shown is read back from the file by `capinfos` after the capture stops; while running it re-reads the growing file.
- Has three ways to stop: a duration target (default 300 s), an optional packet target, and a hard safety timeout at 600 s so a forgotten capture cannot fill the disk.
- Drains the capture tool's error output on a background thread so a full OS pipe buffer cannot freeze the capture, and keeps the last 200 lines for diagnostics.

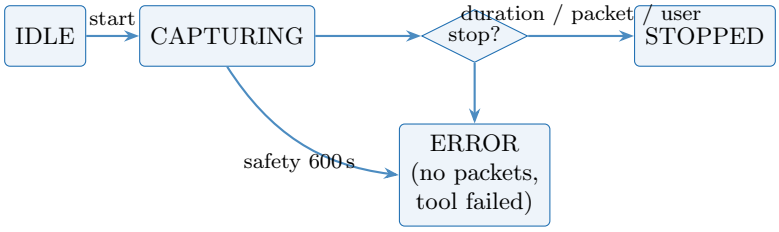


Figure 3: Capture state machine (`capture.py`). `check_and_finalize()` runs on every status poll and is the only place the duration and safety limits are enforced.

Setting	Default
Capture duration	300 s
Packet target	none (a flood is not cut short in a fraction of a second)
Kernel buffer	64 MiB
Safety timeout	600 s
Packet-table page size	100 rows; hard ceiling 50 000 per read

Table 2: Capture defaults, from `backend/capture/capture.py`.

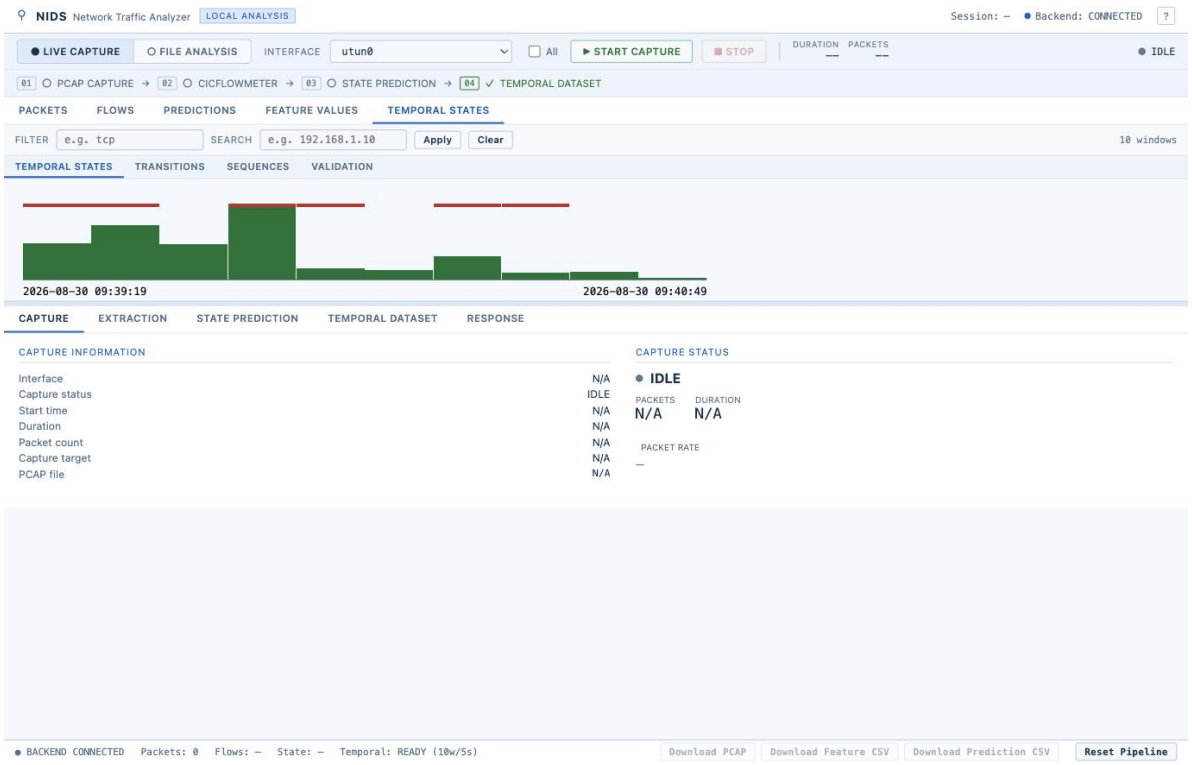


Figure 4: The live-capture panel before a run. Interface picker, Start/Stop, duration and packet counters, and a live packets-per-second sparkline once a capture is running.

3 Stage 2 — Turning packets into flows (CICFlowMeter)

What a "flow" is

A flow is one two-way conversation between two computers, identified by five things: source IP, destination IP, source port, destination port, and protocol. CICFlowMeter reads the whole capture once and, for each flow, produces one row of about 80 columns — packet counts each direction,

byte totals, inter-arrival-time mean/standard-deviation/min/max, TCP-flag counts, header lengths, packet-length statistics, bytes-per-second and packets-per-second, active/idle times. The ANN uses 77 of those columns (the exact CICIDS2017 training schema in `features.py`).

- Uses the `cicflowmeter` Python package (a drop-in for the original Java tool). Two upstream bugs are patched once by `scripts/patch_cicflowmeter.py`.
- Large captures are split on packet boundaries with `editcap` into 20 000-packet chunks and processed in parallel, then the per-chunk CSVs are concatenated. A flow that straddles a cut is counted once per chunk — a small, documented inflation the large chunk size keeps rare.
- Column names drift between CICFlowMeter versions. `match_columns()` aligns any naming style to the 77 training features by reducing each name to a set of canonical tokens (so "Total Fwd Packets" and "Tot Fwd Pkts" match). It raises rather than silently misalign.

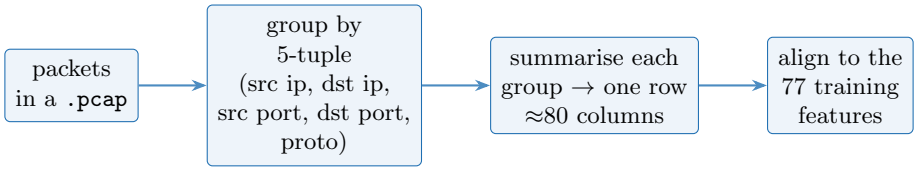


Figure 5: Flow extraction. One conversation becomes one row of numbers.

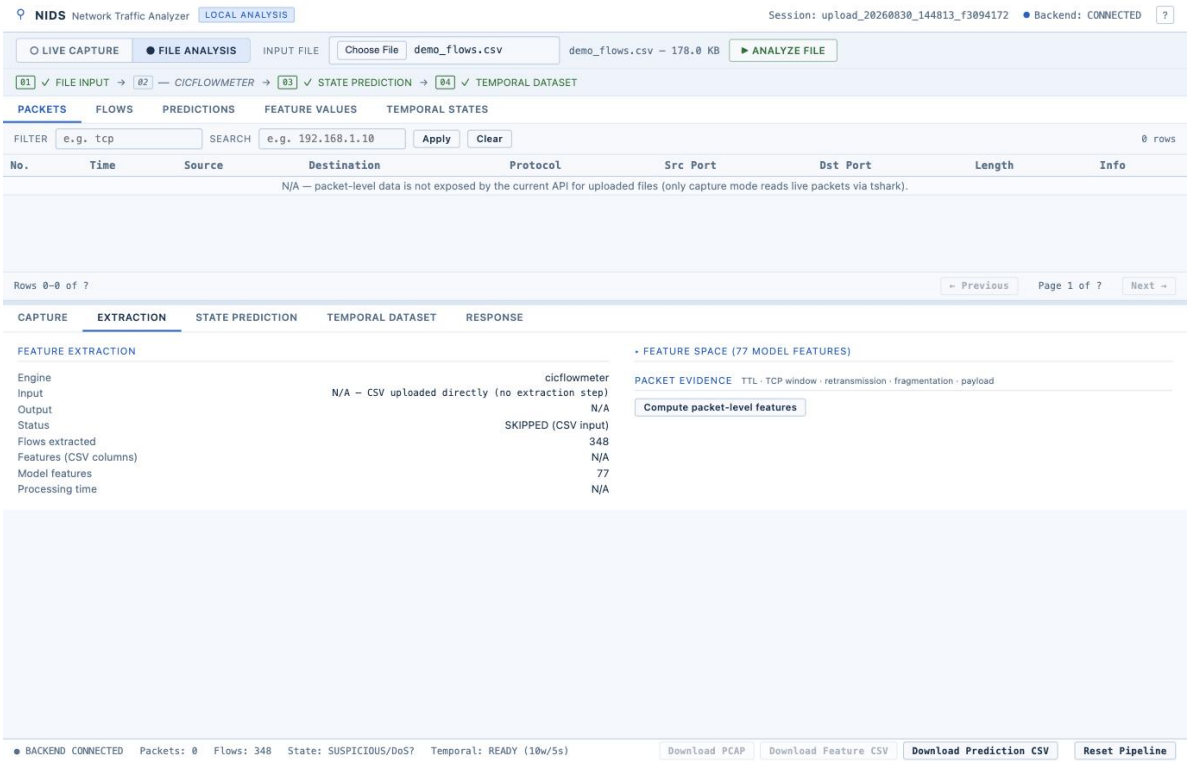


Figure 6: The extraction panel. For an uploaded CSV this step is skipped (the rows are already flows); for a capture it shows flow count, feature count and processing time. "Feature space" lists the 77 model features; "Packet evidence" is the separate Scapy pass below.

3.1 Stage 2b — Packet-level evidence

A second, independent pass (`packet_features.py`, using Scapy) reads the capture directly and computes per-flow *packet*-level signals that flow statistics miss: TTL mean/spread, TCP window behaviour, retransmission count, IP-fragment count, payload-length statistics. It also classifies port-scan

order — whether destination ports were walked `sequential`, `strided` or `randomised`. None of this feeds the ANN; it is shown as corroborating evidence on the dashboard.

4 Stage 3 — The neural-network classifier

What it does, in plain words

- Loads a pre-trained Keras model (`ISAA_ANN.h5`, a small multi-layer network) and its scaler (`minmax.bin`).
- Scales every flow's 77 features to the 0–1 range the model was trained on, then asks the model for four probabilities that sum to 1: how `BENIGN`, `DDoS`, `DoS`, `PortScan` the flow looks.
- The largest probability is the flow's label; the probability itself is the "confidence".
- A flow whose features contain infinities or gaps (a known CICFlowMeter artefact for zero-duration flows) is never guessed — it is marked `INVALID_FEATURES` and excluded from scoring.
- The labels are written back onto the CSV as three new columns (`Current_State` = the ANN call, `Signature_State` = the rule layer's call, `Effective_State` = the merged call) so the file and the on-screen numbers can never disagree.

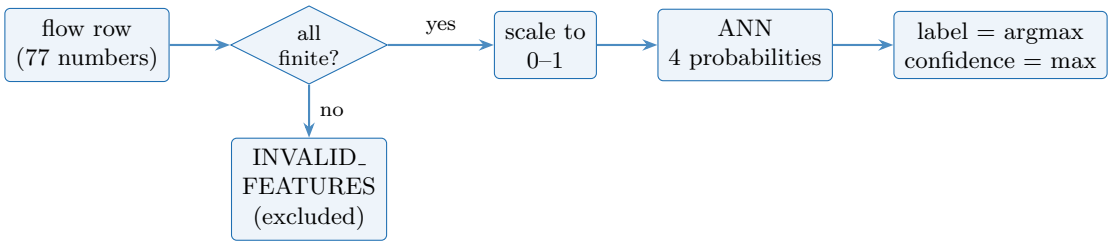


Figure 7: Per-flow scoring (`predict.py`).

4.1 Stage 3b — The signature layer

The ANN sees one flow at a time, so a volumetric DDoS — thousands of small, individually-innocuous flows — can be diluted to "mostly BENIGN". The signature layer (`signatures.py`) looks across the whole flow table at once and fires three deterministic rules:

Rule	Fires when	Marks
Rolling flood	in a 1-second bucket, one victim IP is hit by ≥20 distinct sources, that victim holds ≥50% of the window's flows, the busiest single source is ≤25% of them, and packet rate ≥100/s	every flow in the group → `DDoS`
SYN / half-open flood	a flow has ≥20 SYNs, ACK/SYN ≤0.2, and ≥100 packets/s	BENIGN rows → `DoS`
Port sweep	one source contacts ≥20 distinct destination ports on one destination with mean forward bytes ≤400	BENIGN rows → `PortScan`

Table 3: Signature rules. This layer never overwrites a real ANN call — it only relabels flows the ANN left `BENIGN` or `INVALID_FEATURES`, and it can clear the verdict gate for its class.

4.2 Stage 3c — The verdict gate

The headline a defender reads is one of three: green **BENIGN**, amber **SUSPICIOUS**, red **ATTACK**. The gate (`summarize_states()`) decides which.

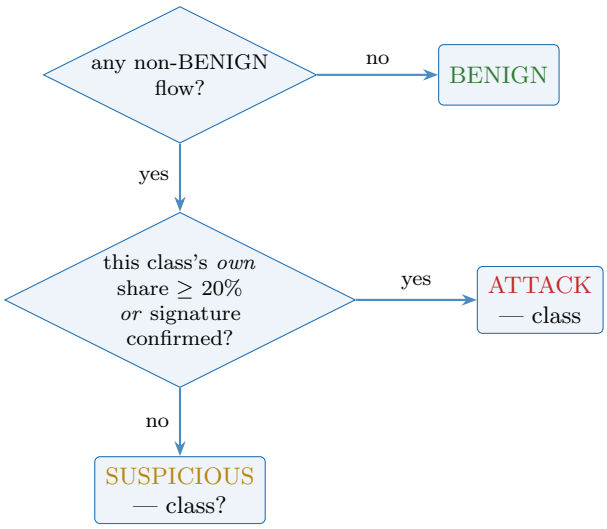


Figure 8: The verdict gate. The test is on the attack class's *own* percentage of scored flows — so a minority DoS is not promoted to red by unrelated DDoS flows in the same capture. Thresholds live in `config/config.json` and can be retuned without touching code.

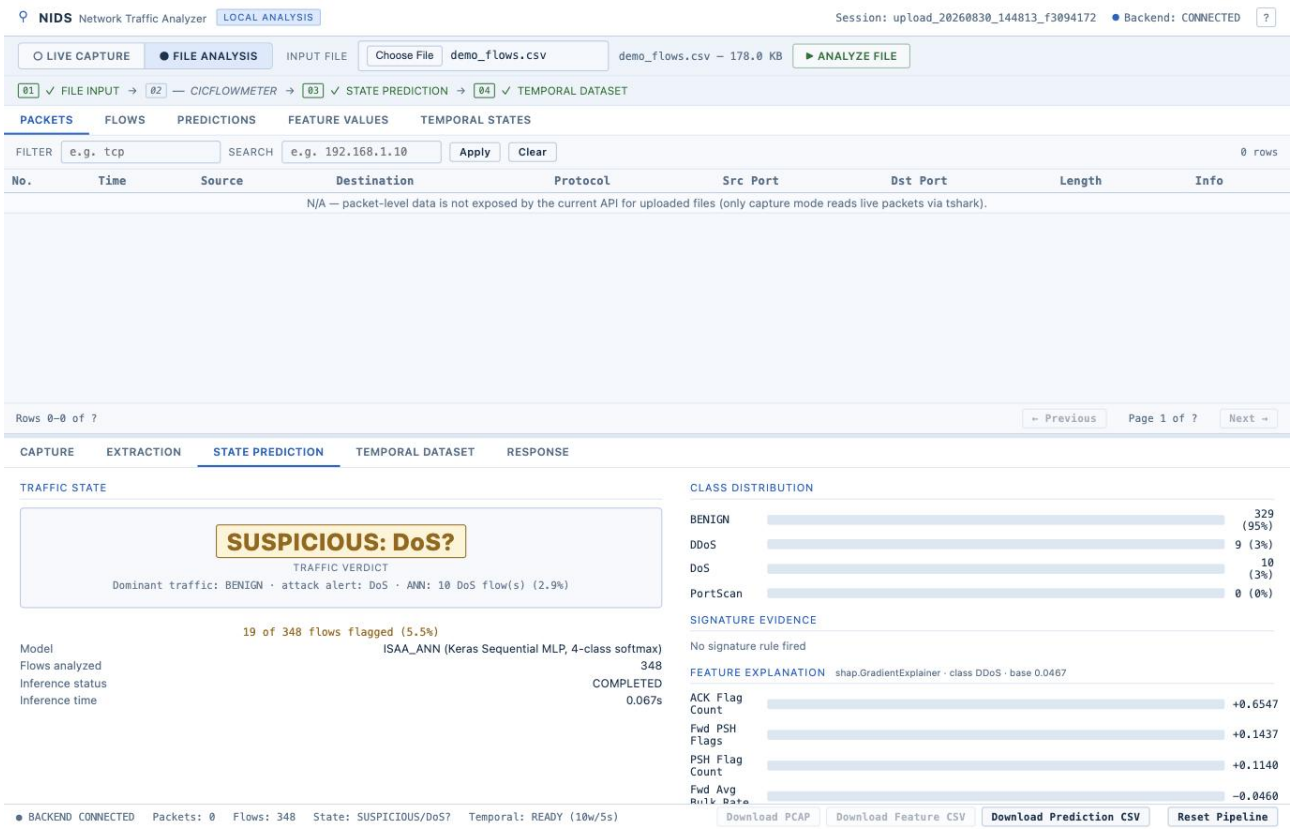


Figure 9: A real verdict. This capture is 95% benign with 10 DoS flows out of 348 (2.9%). Because DoS's own share is well under 20% and no signature fired, the hero reads amber **SUSPICIOUS: DoS?**, not red. The class-distribution bars, "no signature rule fired", and the SHAP feature explanation are all on this one panel.

4.3 Stage 3d — Explaining a prediction

Two mechanisms:

- **Always on:** a single gradient-times-input pass (`explain.py`) gives, per flow, how much each of

the 77 features pushed the model toward its chosen class. The capture-wide top 10 are shown as "driving features".

- **When a background sample exists:** real SHAP values (`shap_service.py`) computed by an async job. The dashboard shows `shap.GradientExplainer` when SHAP ran and "gradient × input fallback — not SHAP" otherwise. The background file `models/ann_shap_background.npy` is present, so real SHAP runs.

4.4 Stage 3e — Classifier quality

The metrics panel (`metrics.py`) reports precision, recall, F1 and support per class, macro/weighted F1, accuracy, an attack-vs-benign block with false- positive rate, and a 4×4 confusion matrix. It uses *real* CICIDS2017 ground-truth labels when a labelled CSV directory is configured (`NIDS_CICIDS2017_DIR`); otherwise it falls back to a *proxy* — how often the ANN agrees with the signature layer — and labels it clearly as "PROXY (not ground truth)".

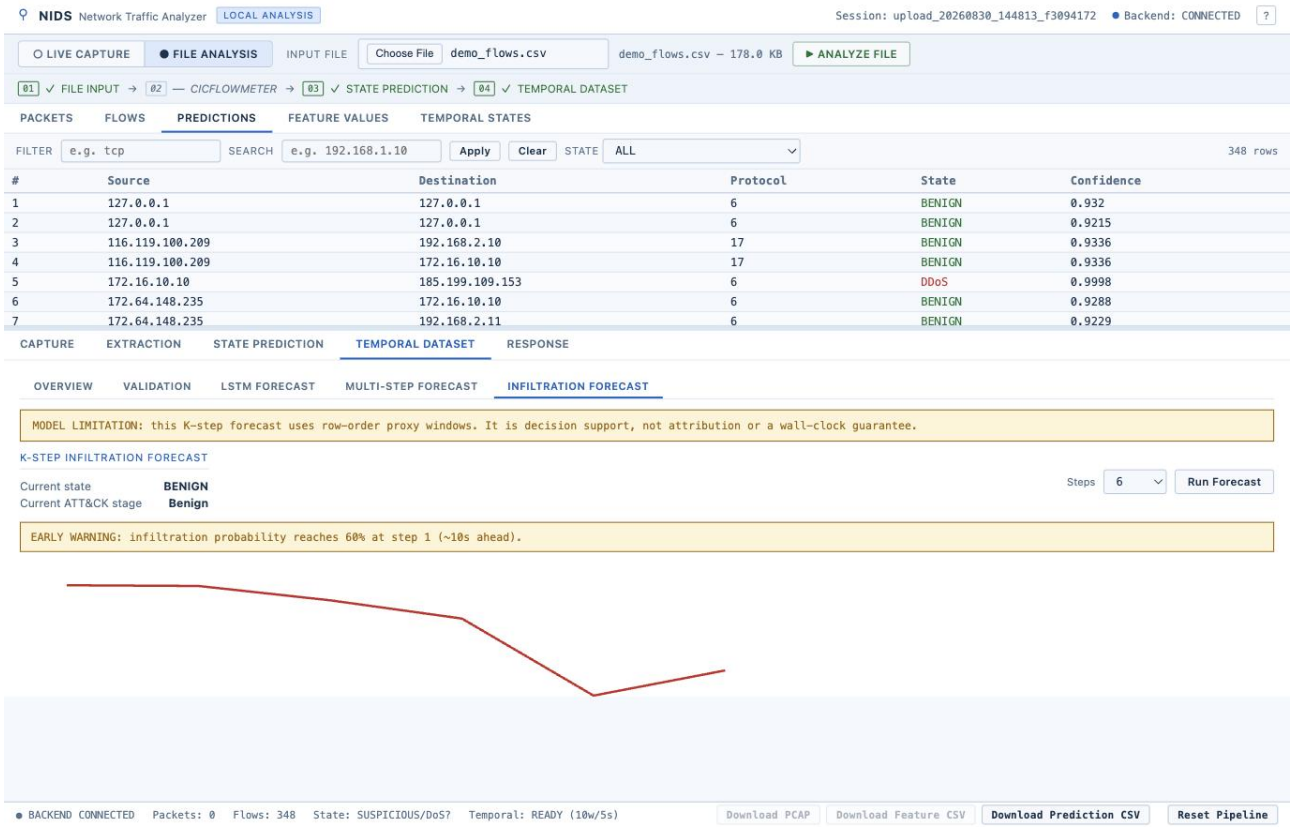


Figure 10: Per-flow predictions: source, destination, protocol, ANN state, confidence. Row 5 here is a DDoS flow scored at 0.9998.

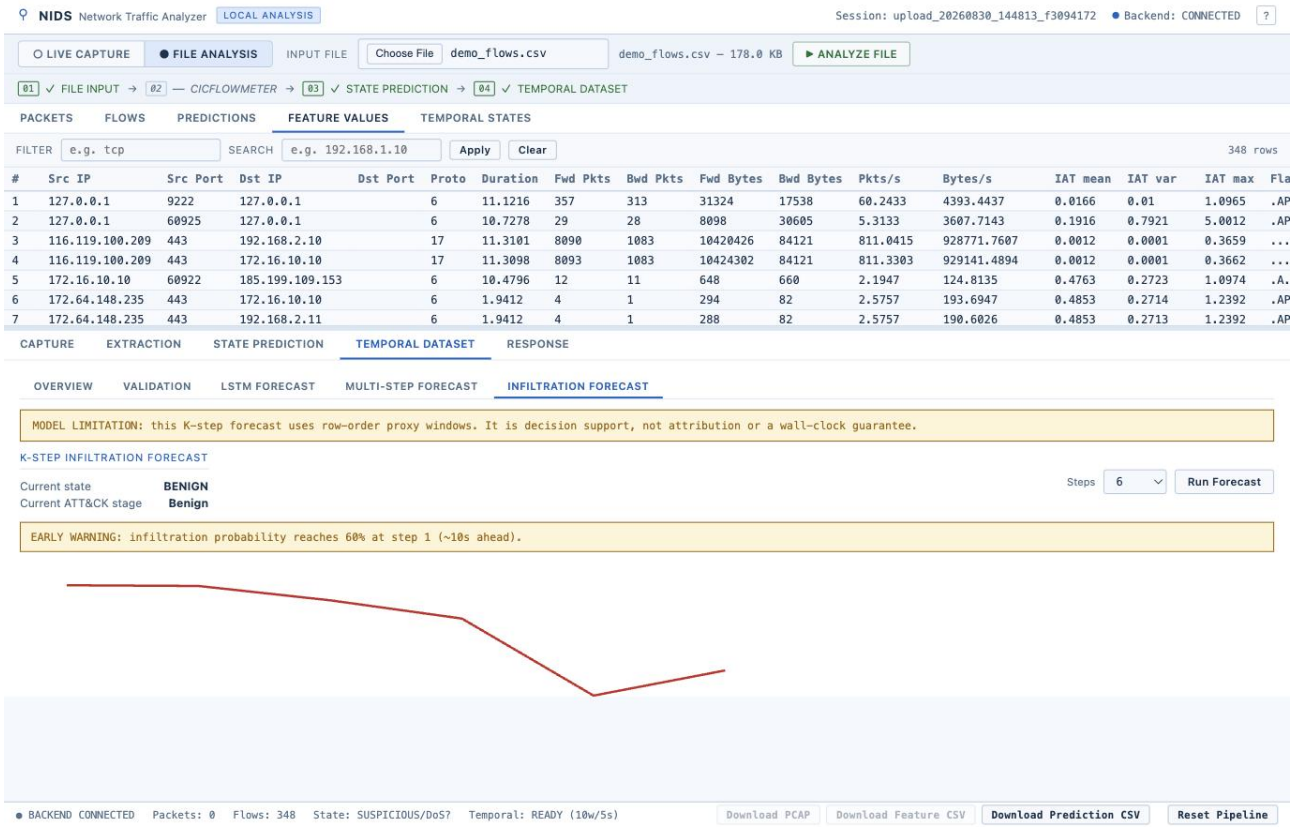


Figure 11: The raw feature values behind each flow — duration, packet and byte counts each direction, rates, inter-arrival timing, and the TCP-flag bitmask.

5 Stage 4 — Building the time-series dataset

What it does, in plain words

- Reads the labelled flow CSV, sorts by real timestamp, and slices time into fixed **10-second windows**.
- For each window it computes a 28-number "state vector" — flow count, unique source/destination IPs and ports, packet and byte totals and rates, TCP-flag totals, inter-arrival-time statistics — plus the dominant class in that window and an `attack_present` flag.
- Builds a table of window-to-window transitions (with the real time gap between them, because empty windows are never invented).
- Slides a length-5 window across the states: five consecutive state vectors are the input, the sixth window's state is the answer.
- Splits the timeline **chronologically** 70/15/15 into train / validation / test — never shuffled — and fits the scaler on the training windows only.

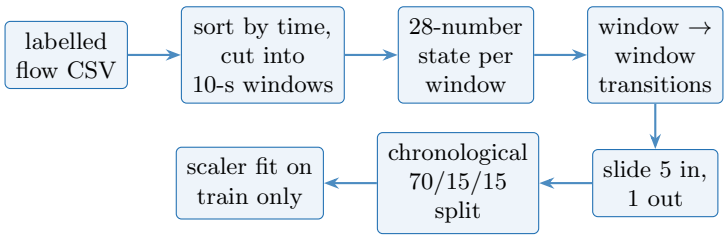


Figure 12: Temporal dataset preparation (`backend/temporal/`). This package never trains a model — it only prepares data.

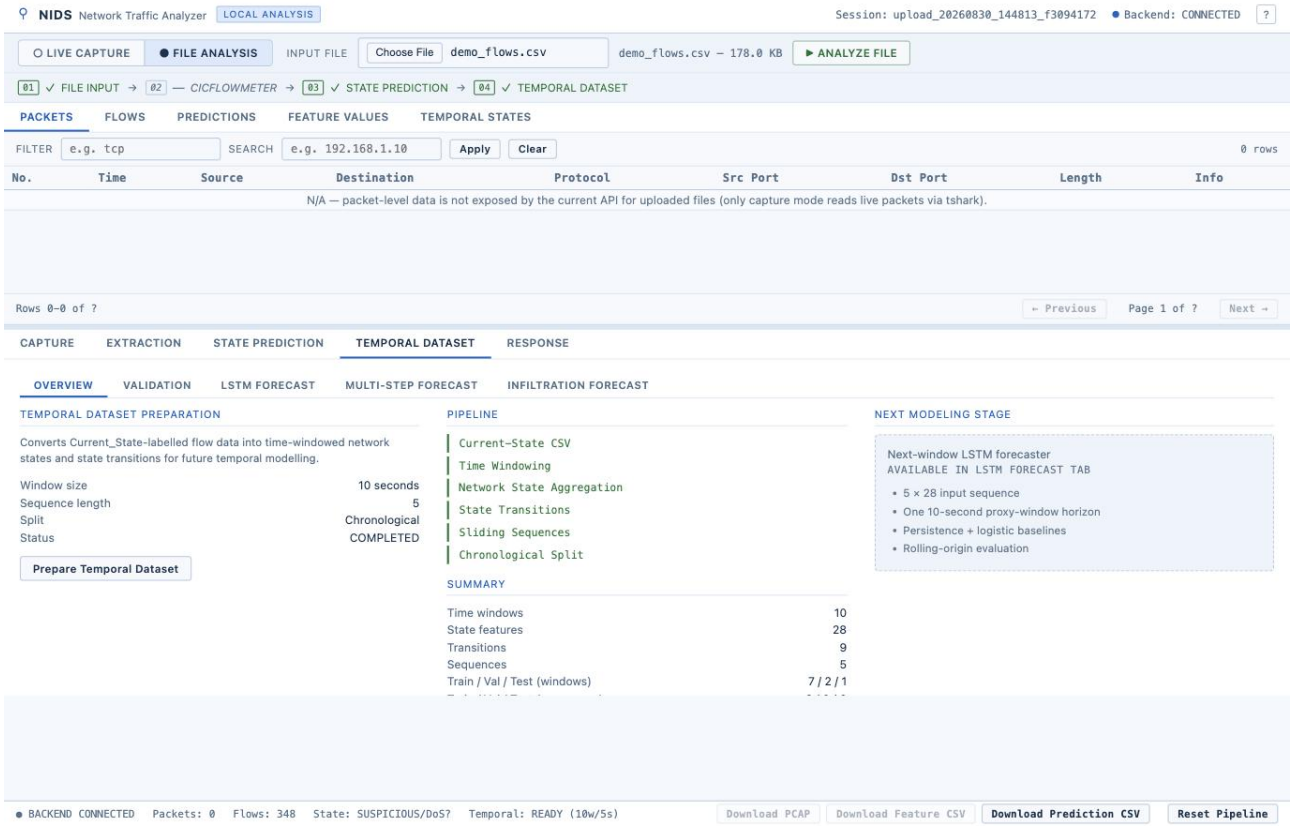


Figure 13: The six-step temporal pipeline on the dashboard, with the run summary: 10 windows, 28 state features, 9 transitions, 5 sequences, a 7/2/1 window split. Short captures produce few windows — the panel says so plainly rather than padding.

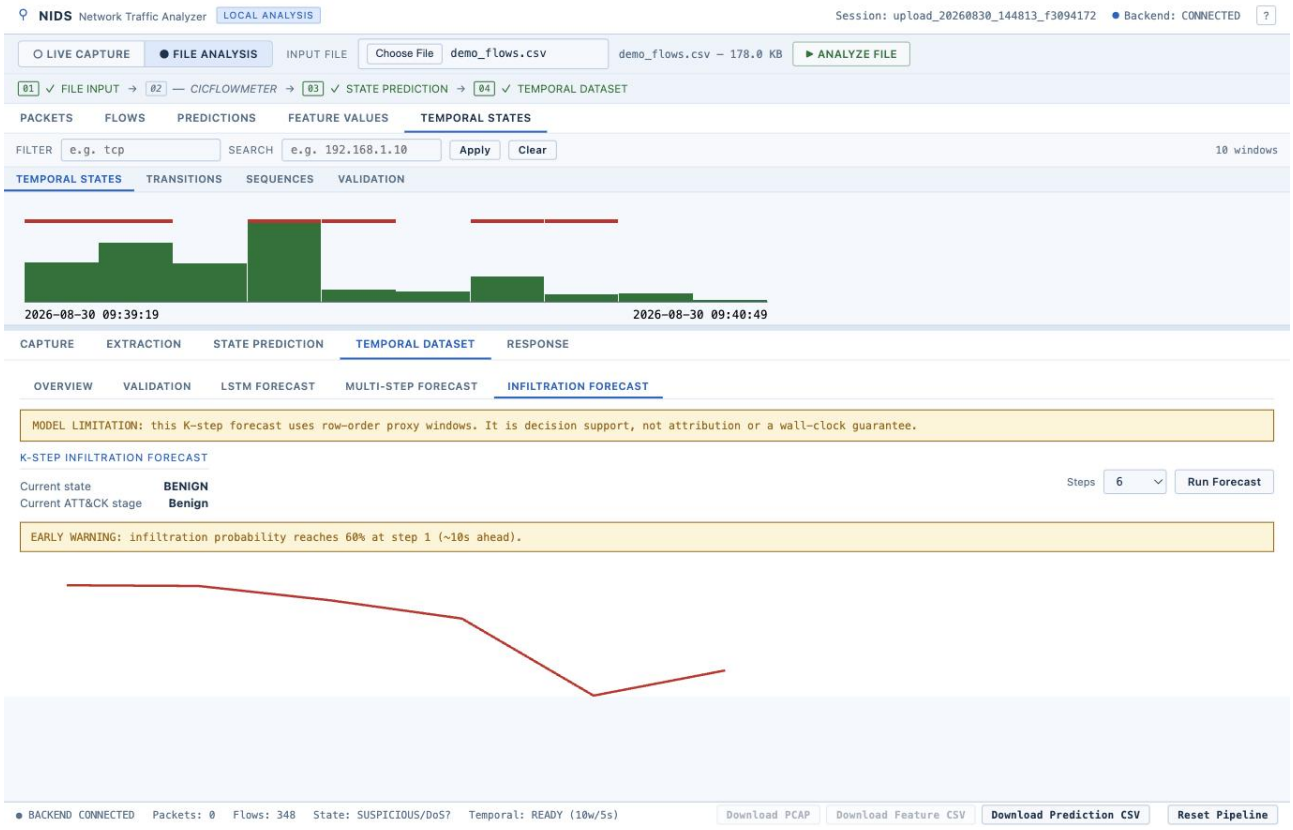


Figure 14: The windows over time. Bar height is flow count, colour is the dominant class, a red tick marks a window with any attack flow. Timestamps are real (here 09:39:19 to 09:40:49).

5.1 Stage 4b — The 10-check validation / leakage audit

Before anyone trusts a forecast, `validate.py` re-checks a prepared dataset read-only and reports each check PASS / WARNING / FAIL:

1. labels present and only the four known classes
2. timestamps parse, and are in order
3. exact-duplicate rows / duplicate IDs
4. missing / infinite cells
5. window duration exact, no overlaps, gaps flagged (expected)
6. all 28 features numeric, non-negative where the name implies it
7. transitions reference real windows and match the actual next state
8. sequence targets are exactly one window after the last input
9. the chronological split really is train $<$ val $<$ test in time
10. **leakage**: no sequence shared across splits, no window shared, the scaler saw training data only, no feature named like the target

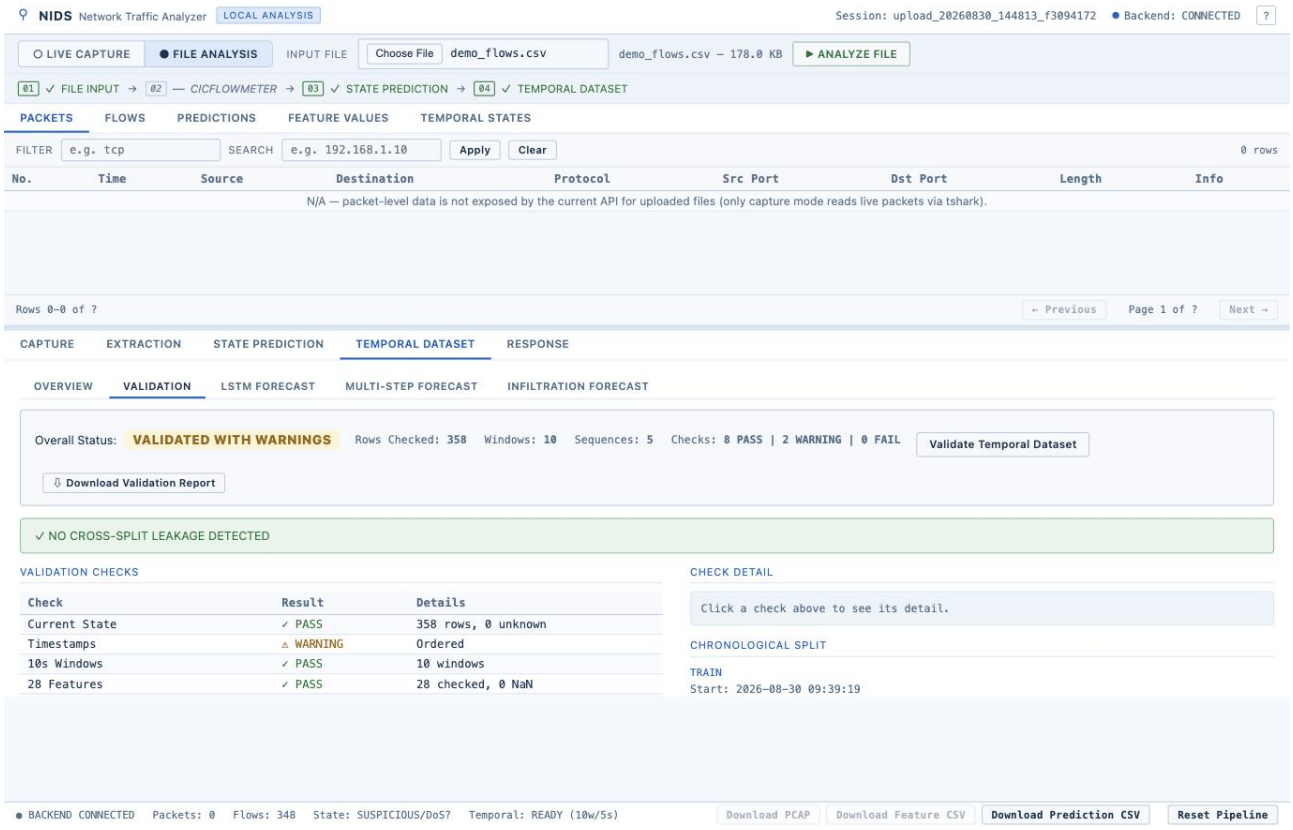


Figure 15: A validation run: 8 PASS, 2 WARNING, 0 FAIL; "NO CROSS-SPLIT LEAKAGE DETECTED". The Timestamps WARNING here is a benign ordering note, not a defect.

6 Stage 5 — Forecasting the next window (one-step LSTM)

What it does, in plain words

- Takes the last five 10-second windows and predicts the *next* window's dominant state and an "attack alert" class, as probabilities.
- The model is a small LSTM: 64 memory units, a shared 32-unit layer, then two 4-way softmax heads. Trained with a fixed random seed for reproducibility.
- Trained on four CICIDS2017 weekday sessions. **Important honesty caveat:** one CSV row is treated as one "proxy second" and a window is 10 *rows*, so horizons are windows, not validated wall-clock seconds.
- Model selection uses three expanding time-ordered folds; the winner is retrained; it is then scored on a strict final holdout against two baselines — *persistence* (predict the last state) and *logistic regression*.
- It is only "activated" (promoted to the live model) if it beats logistic regression on macro-F1 and DDoS recall and keeps the benign false-positive rate within one point. In practice DDoS is absent from the configured sessions, so the honest headline metric is the *validation* split.

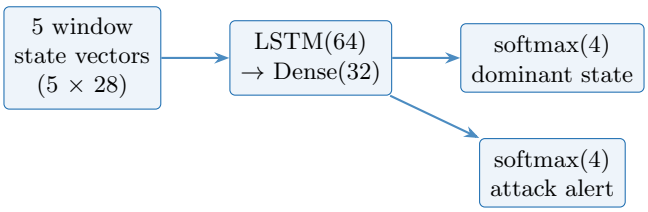


Figure 16: One-step forecaster (`backend/lstm/`). Two heads from one shared body.

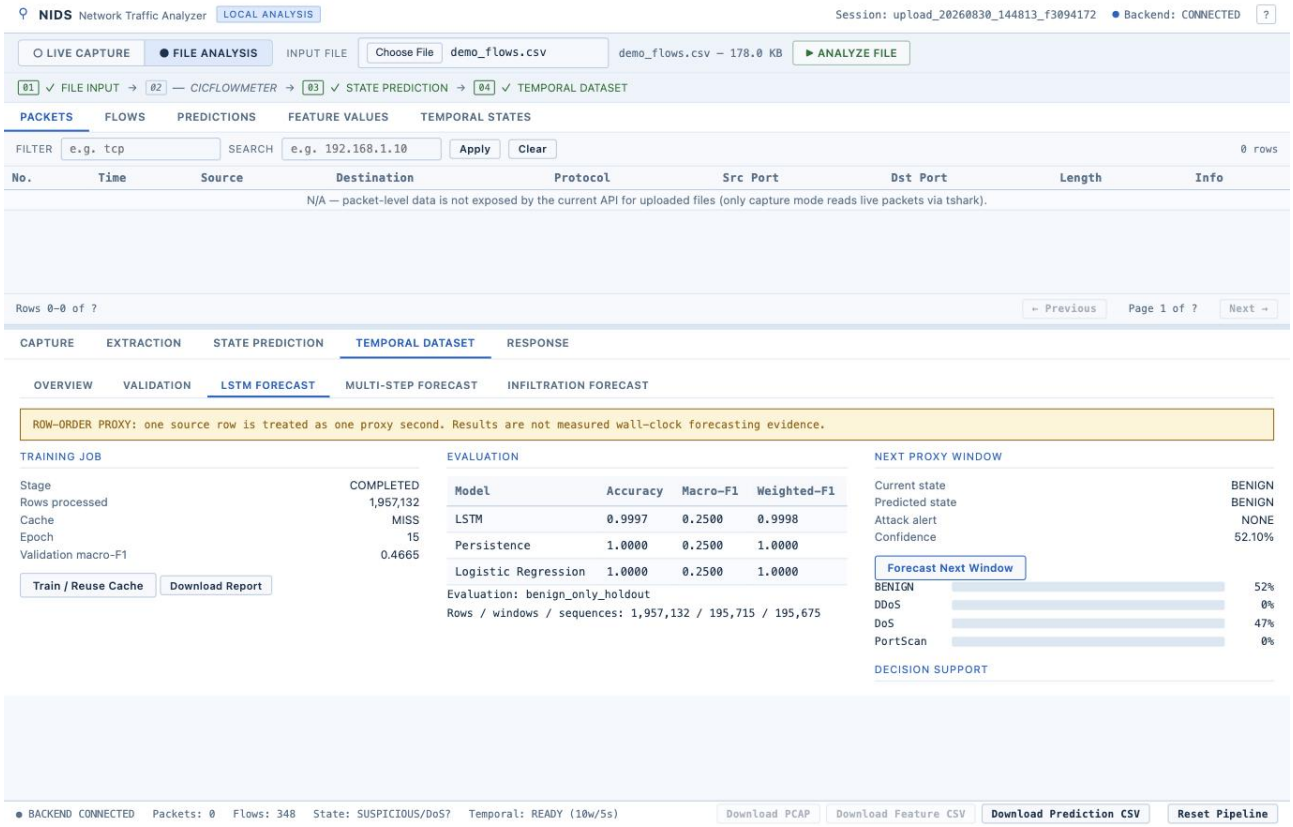


Figure 17: A live one-step forecast: current state BENIGN, predicted BENIGN, attack alert NONE, confidence 52%. The evaluation table compares the LSTM to persistence and logistic-regression baselines; the row-order-proxy caveat is shown at the top of the panel.

6.1 Stage 5b — The multi-step forecast (H1–H6)

A second model (`backend/lstm_multistep/`) predicts the next *six* windows in one shot (heads reshaped to 6×4). It adds:

- an **early-warning threshold** chosen to catch as many attacks as possible while keeping false alarms $\leq 5\%$;
- **onset / lead-time** metrics — for cases where history is all benign and an attack really starts within six windows, how early or late the alarm fires;
- a **degradation table** showing how accuracy falls off as the horizon grows.

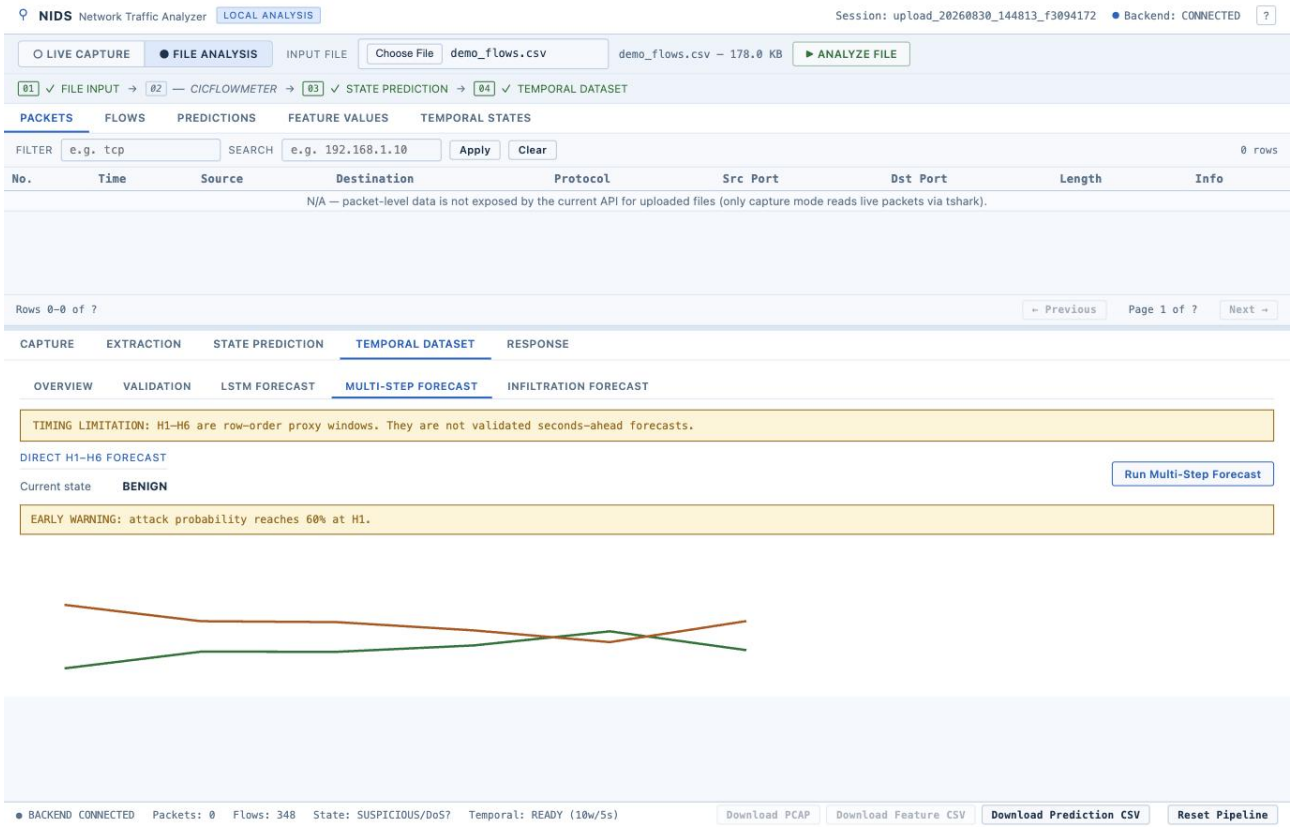


Figure 18: Multi-step: "EARLY WARNING: attack probability reaches 60% at H1", with the per-class probability trajectory across the six horizons.

6.2 Stage 5c — The infiltration forecast (world model)

The world model (`backend/worldmodel/`) differs from the LSTMs by having a **second, regression head that predicts the next window's full 28-number state**. That means it can feed its own prediction back in and keep simulating step after step — an autoregressive "rollout" of K steps (default 6, max 24). It also exposes **attention weights** over the five input windows and maps each predicted state to a kill-chain stage (Recon → . . . → Impact).

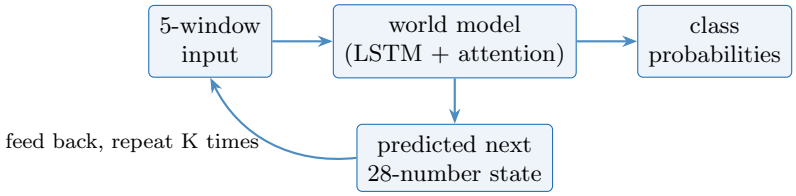


Figure 19: The rollout loop. Steps after the first are conditioned on the model's own imagined states, not on real data — hence "forward-simulating world model" rather than "one-step classifier".

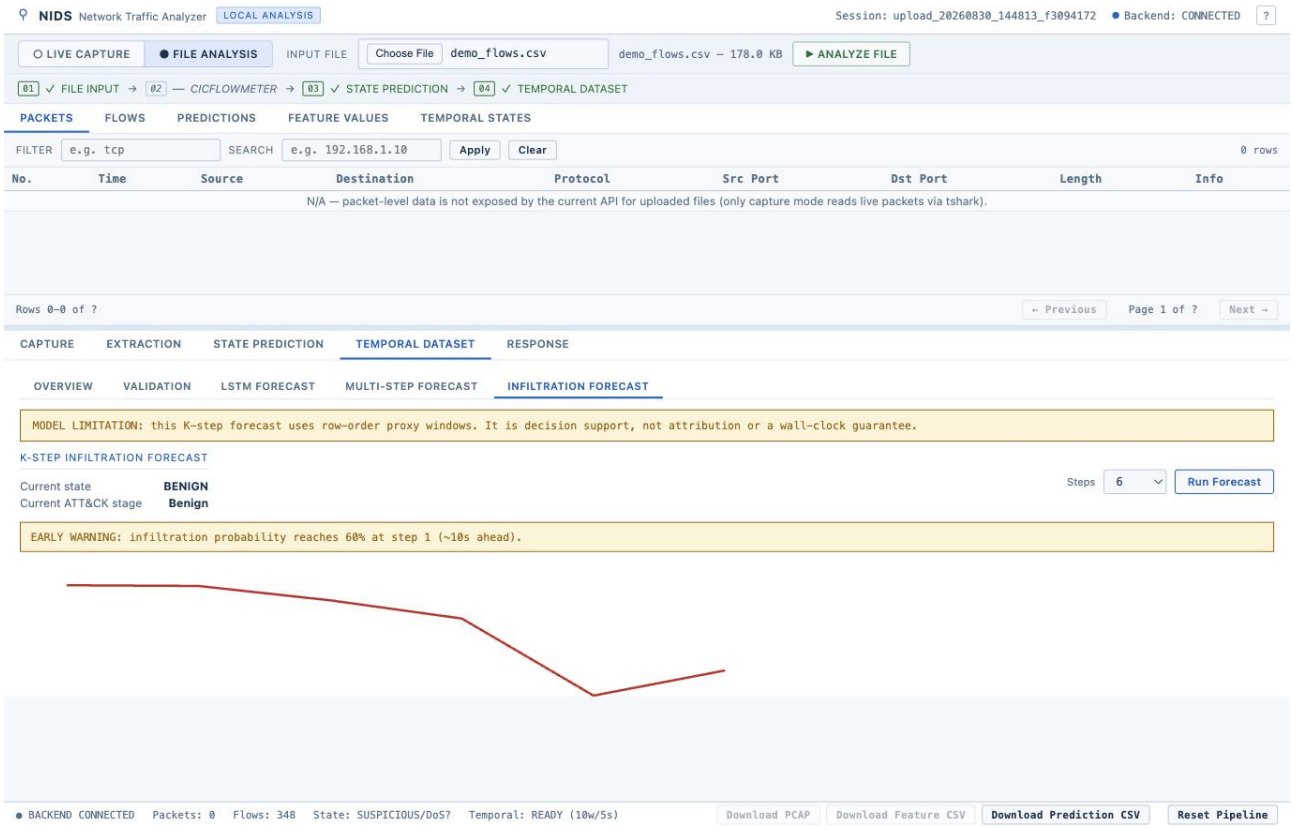


Figure 20: Infiltration forecast: "infiltration probability reaches 60% at step 1 (∼10 s ahead)", with the probability curve over the K steps and the current ATT&CK stage.

7 Stage 6 — MITRE ATT&CK context

The mapper (`mitre/mapper.py`) is deliberately restrained. It holds a fixed table — `PortScan` → (T1595 Active Scanning, T1046 Network Service Discovery); `DDoS` → (T1498 and sub-techniques); `DoS` → (T1499) — and, given the current state, the next-state probabilities and the latest window's raw numbers, it produces technique *candidates* with:

- a `mapping_status` of `POSSIBLE` or `INSUFFICIENT_EVIDENCE`;
- a `mapping_confidence` of 0.25, 0.55 or 0.65 — explicitly typed `deterministic_heuristic_not_calibrated`;
- human evidence sentences built from thresholds (port diversity, SYN volume, packet rate);
- a severity (high / medium / informational), an operator playbook and an "evidence needed" list;
- the offline ATT&CK version (Enterprise v19.1), pinned and provenance-checked.

Every candidate carries the caveat that network-flow evidence alone cannot confirm intent or host behaviour.

Listing 1: `nids mitre map --state PortScan --prob PortScan=0.7 --prob BENIGN=0.3` (abridged)

```
current_state: PortScan
predicted_next_state: PortScan
mapping_status: INSUFFICIENT_EVIDENCE
mitre_candidates:
  - technique_id: T1046   (Network Service Discovery, tactic Discovery)
    mapping_confidence: 0.25
    mapping_confidence_type: deterministic_heuristic_not_calibrated
    evidence: [ observed current network state is PortScan,
                LSTM next-state probability for PortScan is 0.700 ]
```

```
  - technique_id: T1595   (Active Scanning, tactic Reconnaissance)
    mapping_confidence: 0.25
attack_version: 19.1
severity: medium
```

8 Stage 7 — The dashboard, panel by panel

Panel	What it shows
Live capture bar	interface picker, Start/Stop, duration + packet counters, packets-per-second sparkline, capture status dot.
File analysis bar	pick a `.pcap`/`.pcapng`/`.csv`, Analyze; offline, its own state machine.
Pipeline strip	four stages (Capture / CICFlowMeter / State prediction / Temporal), coloured by progress.
Packets / Flows / Predictions / Feature values / Temporal states	the left table, one tab each.
CAPTURE detail	every capture field, plus a hero block and (live) the sparkline.
EXTRACTION detail	input/output, flow and feature counts, the 77-feature list, and the "compute packet-level features" button.
STATE PREDICTION detail	the verdict hero, class distribution, signature hits, feature explanation, a flows-over-time timeline, and a classifier-metrics panel with a confusion matrix.
TEMPORAL DATASET detail	five inner tabs: Overview, Validation, LSTM forecast, Multi-step forecast, Infiltration forecast.
RESPONSE detail	the local response module (see below).
Status bar	backend state, packet/flow counters, the current verdict, temporal readiness, and export buttons.

Table 4: Dashboard map. Element IDs and the exact render function for each are in Part II.

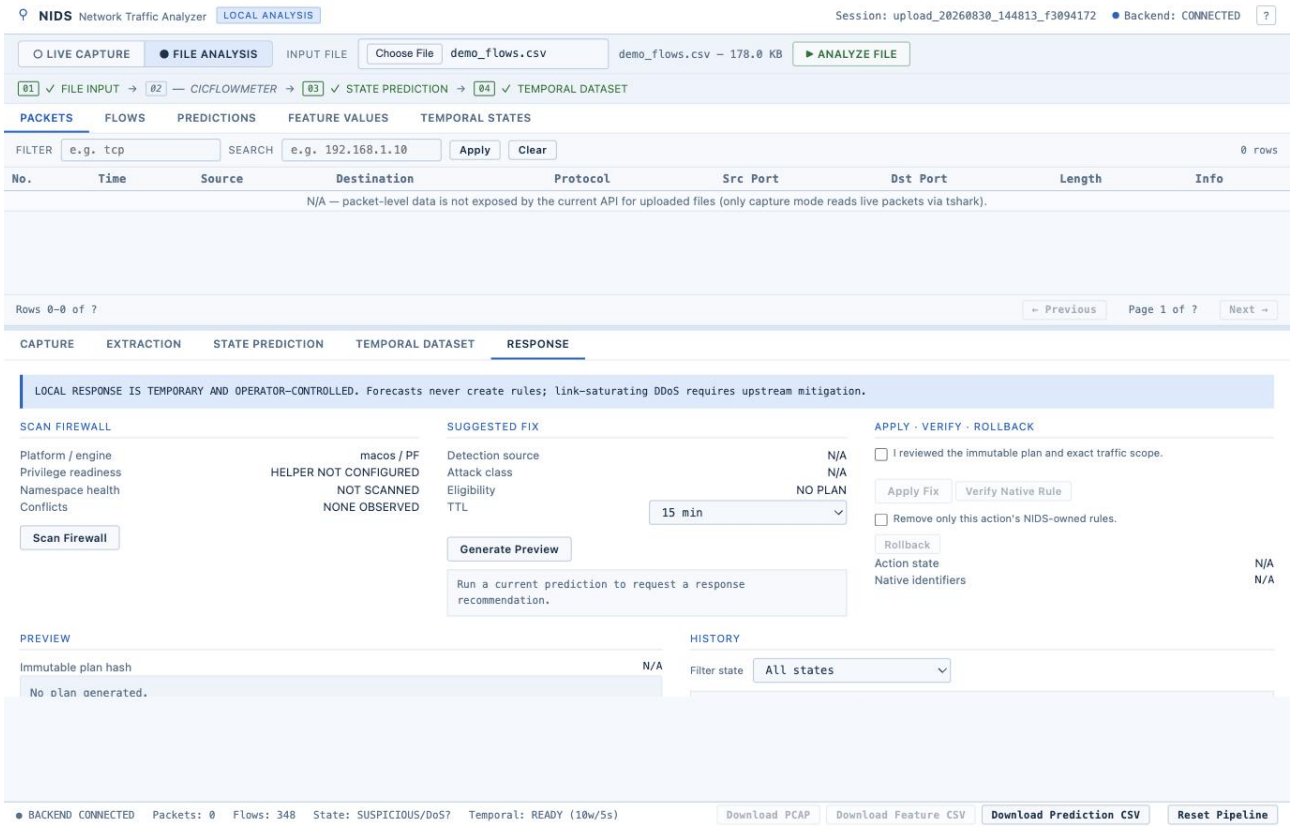


Figure 21: The RESPONSE panel — an early active-response arm. It scans the local firewall (here `macos / PF`), turns a detection into a *suggested* rule with a TTL, and gates apply / verify / rollback behind an explicit operator checkbox. The banner is blunt: "LOCAL RESPONSE IS TEMPORARY AND OPERATOR-CONTROLLED. Forecasts never create rules; link-saturating DDoS requires upstream mitigation." Protected addresses are allow-listed in `config.json`.

9 Stage 8 — The command line

Everything the dashboard does is also a `nids` subcommand (each is a thin wrapper over the same service function the matching `/api/` route calls).

Command	Does
`serve`	run the API + dashboard on `:8765`.
`interfaces`	list capturable interfaces.
`capture start|packets|stat`	foreground capture; paginated packet read; `capinfos` of a pcap.
`extract <pcap>`	pcap → feature CSV.
`predict <csv>`	per-flow ANN prediction.
`predict-metrics`	precision/recall/F1/FPR + confusion matrix.
`explain --model ann|lstm|hgb <values.npy>`	submit a SHAP/attribution job.
`temporal prepare|validate`	build / audit a temporal dataset.
`lstm train|status|forecast|evaluate|report`	one-step forecaster.
`multistep dataset|train|evaluate|benchmark`	direct H1–H6.
`worldmodel train|status|forecast`	K-step infiltration forecaster.
`benchmark`	LSTM vs logistic-regression table.
`mitre map --state S --prob C=V`	ATT&CK candidates for a forecast.
`pipeline show`	on-disk inventory of datasets, model versions, reports.

Table 5: The `nids` CLI. Exit codes: 0 ok, 2 usage, 3 missing artifact/dataset, 4 runtime failure, 130 interrupted.

Listing 2: `nids pipeline show` on the live repository

```
lstm_forecaster_versions: [ v1-3a5264a499ed, v1-6b89455037d9, v1-9f7373e0b3ad ]
lstm_forecaster_active: True
lstm_multistep_versions: [ v1-6d80927e0885 ]
worldmodel_trained: True
reports: { lstm_evaluation: True, multistep_evaluation: True, multistep_benchmark: True }
shap_backgrounds: { ann: True }
```

Listing 3: `nids benchmark` — one-step LSTM vs logistic regression, validation split

```
model_version: v1-3a5264a499ed   headline_split: validation
                       LSTM        Logistic Regression
macro_f1               0.4623      0.4796
weighted_f1            0.9978      0.9988
attack_f1              0.8506      0.9191
attack_recall          0.9328      0.9076
attack_false_pos_rate  0.00188     0.00048
```

The LSTM's edge on this split is *attack recall* (it misses fewer onsets), not headline F1 — on macro-F1 and false-positive rate the logistic baseline is ahead. This is stated plainly in the evaluation report and is why activation rarely fires.

10 Project history

Phase	What landed
Phase 0	The original student project: a Keras ANN + scaler, three notebooks, a `tcpdump` wrapper. No backend, no automation — and the prediction notebook never applied the scaler, so its committed output was the same four numbers for every row. This is the baseline the whole project fixed.
Phase 1	Productisation: the end-to-end capture→extract→ANN pipeline, a FastAPI state machine, the dashboard, and Phase 3 LSTM forecasting + ATT&CK context.
Phase 2	Cross-platform capture (macOS friendly names, one-command setup) and the direct H1–H6 multi-step forecaster; a warnings clean-up.
Phase 3	Explainability (gradient×input), packet-level features, and the first product PDF; an "honest overall state" rework (dominant-class + attack ratio).
Phase 4	Forecasting hardening, multi-interface capture, the logistic-regression benchmark, and a unified `nids` CLI + an async SHAP service.
Phase 5	A run of honesty/robustness fixes: the status bar made to honour the confidence gate; cross-platform peak-RAM measurement; and the per-class percentage verdict gate (a minority class no longer triggers a red "ATTACK" banner). The world model and its dashboard tab were added here.

Table 6: From a broken single notebook to an offline capture→forecast pipeline with a CLI, a SHAP service and a restrained ATT&CK layer.

Part II — Function index

Every public function and class, grouped by file, one line each. Use this to find the exact place a behaviour lives.

backend/capture/capture.py

Name	Location	What it does
`list_interfaces`	`c.py:180`	list real capturable NICs (dumpcap/tcpdump -D), friendly names on macOS.
`start_capture`	`c.py:518`	build and launch the dumpcap/tcpdump command line; 0.5 s sanity wait.
`stop_capture`	`c.py:760`	user stop; SIGINT then escalate; safe if already exited.
`check_and_finalize`	`c.py:799`	on every poll: detect self-exit / duration / 600 s safety, finalize.
`_finalize`	`c.py:703`	shared stop tail: stamp end time, parse drops, read real packet count.
`read_packets`	`c.py:444`	paginated per-packet rows via a tshark frame-range filter.
`_capfile_stats`	`c.py:360`	ask capinfos for real packet count + duration.
`validate_and_stat_pcap`	`c.py:393`	strict pcap check for uploads (raises on 0 packets).
`_parse_capture_drops`	`c.py:673`	pull received/dropped counts from the tool's exit summary.
`CaptureSession`	`c.py:260`	dataclass: process handle, path, targets, results, stderr tail.

backend/extraction/

Name	Location	What it does
`run_extraction`	`extract.py:28`	replay a pcap through CICFlowMeter once -¿ one feature CSV.
`run_parallel_extraction`	`parallel_extract.py:69`	split big pcaps with editcap, extract chunks in parallel, concat.
`extract_packet_features`	`packet_features.py:47`	per-flow TTL / TCP-window / retransmission / fragment / payload stats (Scapy).
`port_scan_signature`	`packet_features.py:142`	the order dst ports were first contacted; classify the sweep.
`_classify_port_order`	`packet_features.py:198`	sequential / strided / randomised / none.
`packet_features_summary`	`packet_features.py:216`	JSON bundle for the PACKET FEATURES panel.

backend/prediction/features.py

Name	Location	What it does
TRAINING_FEATURES	features.py:25	the ordered list of 77 feature-column names (do not reorder).
match_columns	features.py:105	map any CSV's columns to the 77 by canonical token signature; raises on a miss.
_canonical_signature	features.py:95	reduce a column name to a sorted tuple of canonical tokens.
is_id_or_label_column	features.py:136	true for identifier/label columns (never features).

backend/prediction/predict.py

Name	Location	What it does
predict_csv	predict.py:250	main: read CSV, score every flow, run signatures, write labels back, return the result dict.
summarize_states	predict.py:72	per-class counts -¿ headline verdict + confidence, applying the 20% gate.
_load_artifacts	predict.py:216	load ISAA_ANN.h5 + minmax.bin once (cached).
_flow_feature_values	predict.py:159	per row: 21 curated feature values + a TCP-flag bitmask.
CLASS_NAMES	predict.py:43	[BENIGN, DDoS, DoS, PortScan] — softmax index order.
MIN_ATTACK_RATIO_x	predict.py:68	verdict-gate thresholds, read from config/-config.json.

backend/prediction/signatures.py

Name	Location	What it does
flow_signatures	signatures.py:66	whole-table rule layer: rolling flood, SYN flood, port sweep -¿ per-row states + summary.
_port_access_pattern	signatures.py:49	classify how a set of dst ports was walked.
DDOS/DOS/PORTSCAN_x	signatures.py:29	the rule thresholds (fan-in 20, pkts/s 100, SYN 20, ACK-ratio 0.2, ports 20, bytes 400).

backend/prediction/ (explain, SHAP, challenger, metrics)

Name	Location	What it does
attribute	explain.py:29	gradient x input saliency for the ANN (one backward pass).
driving_features	explain.py:74	capture-wide top 10 features by mean absolute contribution.

Name	Location	What it does
tree_explanation / gradient_explanation	shap_service.py:56/78	exact SHAP for the tree challenger / a Keras model.
gradient_input_fallback	shap_service.py:99	used when real SHAP throws; is_shap:false.
ExplanationJobs.submit	shap_service.py:137	async attribution job store (thread pool, cache by hash).
submit_explanation	explanation_runtime.py:33	wire model kind to the right explainer; needs a background .npy.
fit_challenger / train_flow_challenger	flow_challenger.py:32/63	HistGradientBoosting alternative model (not auto-promoted).
evaluate_ann_metrics	metrics.py:167	real CICIDS2017 ground-truth metrics, or None.
proxy_agreement_metrics	metrics.py:231	fallback: ANN-vs-signature agreement (not ground truth).
compute_metrics	metrics.py:136	precision/recall/F1/support, macro/weighted F1, confusion matrix.

backend/temporal/

Name	Location	What it does
prepare_temporal_dataset	temporal_dataset_preparator.py:81	orchestrator: CSV -¿ windows -¿ states -¿ sequences -¿ split -¿ files.
parse_timestamps	windowing.py:31	read the time column, tracking unparseable rows honestly.
assign_windows	windowing.py:77	label each flow with its 10-second slot.
build_temporal_states	state_builder.py:20	one window: 28-number state vector + dominant class + attack_present.
build_state_transitions	sequence_builder.py:19	windows -¿ next-window pairs with real time delta.
build_sequences	sequence_builder.py:55	slide: N windows in, 1 out; strictly chronological.
chronological_split	splitting.py:22	first 70% / next 15% / last 15% by time.
fit_scaler_on_train	splitting.py:51	MinMaxScaler on training windows only.
run_all_checks	validation.py:73	the 7 build-time gate checks.
validate_temporal_dataset	validate.py:544	the 10-check read-only audit incl. O(n) leakage detection.
STATE_FEATURE_NAMES	schema.py:72	the 28-element state-vector schema.

backend/lstm/ (one-step forecaster)

Name	Location	What it does
train_forecaster	training.py:283	full run: windows -¿ 3-fold selection x 3 loss recipes -¿ retrain -¿ holdout vs baselines -¿ report.
build_model	training.py:83	LSTM(64)-¿Dropout(0.3)-¿Dense(32)-¿ two softmax(4) heads.

Name	Location	What it does
forecast_latest	training.py:547	run the active model on the recent 5 windows -¿ next-window probabilities + MITRE map.
_load_recent_windows	training.py:495	last 5 contiguous prepared windows to forecast from (shared by all three forecasters).
rolling_origin_windows	training.py:103	3 expanding time-ordered folds for model selection.
_persistence / _logistic	training.py:229/238	the two baselines on the holdout.
evaluate_predictions	evaluation.py:11	the metric battery: 4-class macro-F1 and binary attack-vs-benign.
cicids_ground_truth_state	dataset.py:35	map a CICIDS2017 Label to BENIGN/DDoS/DoS/PortScan or None.
prepare_session_windows	dataset.py:275	build (or cache) proxy windows for one CICIDS2017 session (10 rows = 1 window).
evaluate_saved_artifact	rigorous_evaluation.py:442	honest evaluation + 8-check leakage audit; writes the report.
start_training	jobs.py:56	spawn the training process; status file polled by the UI.

backend/lstm_multistep/ (direct H1–H6)

Name	Location	What it does
train_multistep	training.py:206	build dataset -¿ train the 6x4-head model -¿ threshold selection -¿ per-horizon eval -¿ activation gate.
build_model	training.py:50	LSTM(64)-¿Dense(32)-¿ Dense(24)-¿Reshape(6,4)-¿softmax, twice.
forecast_latest	training.py:304	one forward pass -¿ 6 horizons; earliest horizon whose attack prob ¿= threshold.
build_multistep_sequences	dataset.py:24	slide 5 history + 6 target windows.
split_session_windows	dataset.py:82	5 train sessions, 2 validation, Friday-DDoS cut 60/20/20 with embargo.
select_early_warning_threshold	evaluation.py:22	pick the alarm cutoff: max attack-F1 s.t. FPR ¡= 5%.
onset_metrics	evaluation.py:41	early / on-time / late detection and mean lead windows.
degradation_table	evaluation.py:70	how accuracy/F1 decay from H1 to H6.

backend/worldmodel/ (K-step infiltration)

Name	Location	What it does
build_world_model	model.py:26	LSTM(64, return_sequences) + attention over 5 steps + two heads: class softmax(4) and next-state Dense(28).
rollout	model.py:52	predict next state + feature vector, slide it onto the input, repeat K times.

Name	Location	What it does
train_world_model	training.py:138	build 5-¿1 pairs with a 28-d regression target; loss weights 1.0 / 0.3; macro-F1 ¿= 0.40 =¿ VALIDATED.
forecast	engine.py:102	roll K steps -¿ per-step infiltration probability, risk level, kill-chain stage, top features, attended windows.
_feature_attribution	engine.py:74	gradient x input over the 28 state features for the "not benign" score.
state_to_stage / presentation_state	killchain.py:40/44	class state -¿ kill-chain rung + tactic id + ladder.
risk_level	killchain.py:64	¿=0.80 high, ¿=0.50 medium, else low.

backend/mitre/mapper.py

Name	Location	What it does
map_forecast	mapper.py:172	state + probabilities + features -¿ technique candidates + severity + playbook.
_behavior_evidence	mapper.py:99	turn raw window numbers into human evidence sentences (fixed thresholds).
_guidance	mapper.py:138	fixed playbooks per state (DDoS/DoS high, PortScan medium, else informational).
AttackMetadataStore.load	mapper.py:33	read + strictly validate the offline ATT&CK v19.1 subset.
STATE_TECHNIQUES	mapper.py:76	PortScan-¿(T1595,T1046); DDoS-¿(T1498,.001,.002); DoS-¿(T1499).

backend/api/main.py (endpoints)

Name	Location	What it does
GET /api/pipeline	main.py:377	the whole capture-¿predict snapshot.
POST /api/capture/start|stop	main.py:145/170	start / stop a live capture.
GET /api/capture/status|packets	main.py:212/219	live status; paginated packet page.
POST /api/extract|predict	main.py:252/326	background extraction / prediction jobs.
GET /api/predict/metrics	main.py:342	ground-truth or proxy classifier metrics.
POST /api/upload	main.py:410	offline file analysis (pcap or csv), own state machine.
POST /api/temporal/prepare|validate	main.py:519/640	build / audit a temporal dataset.
POST /api/lstm/forecast|.../multistep	main.py:785/793	one-step / H1–H6 forecast.
POST /api/worldmodel/forecast	main.py:818	K-step infiltration forecast.
GET /api/benchmark	main.py:833	LSTM vs logistic-regression table.

frontend/app.js (render functions, grouped)

Name	Location	What it does
predVerdict	app.js:1138	shared display verdict: none / benign / suspicious / attack from confidence.
renderPredictionTab	app.js:1148	the verdict hero, dominant/alert sub-line, flagged-flows line, sub-panels.
renderClassDistribution	app.js:1422	per-class bar with count and % of flows.
renderSignatureHits	app.js:1220	one row per fired signature rule.
renderDrivingFeatures	app.js:1239	SHAP job result if present, else gradient x input; signed bars.
renderClassifierMetrics	app.js:1091	ground-truth vs proxy badge, per-class table, confusion heat-map.
renderPredictionFlowsTimeline	app.js:1350	benign/attack flows per minute + mean confidence line (inline SVG).
renderTemporalTab / renderValidationTab	app.js:1439/1661	the 6-step strip + summary; the 10-check grid + leakage banner.
renderLstmTab	app.js:1492	training job rows, evaluation table, next-window probabilities, MITRE decision support.
renderMultistepTab / renderWorldModelTab	app.js:1548/1594	early-warning banner + trajectory chart + per-horizon / per-step details.
renderCaptureTab / renderCaptureRateSparkline	app.js:887/1401	capture fields + hero; client-side packets/sec sparkline.
startPolling / refreshPipeline	app.js:347/313	the 1.2 s master poll of /api/pipeline.

Part III — Attacks, and how this app handles them

11 The three levers

Lever	App machinery	Output
FLAG	77 flow features -¿ ANN; the signature layer; packet-level features; the verdict gate; the MITRE mapper; SHAP.	per-flow label, capture verdict, ATT&CK technique, "why".
PREDICT	10-second windows -¿ one-step LSTM + direct H1–H6 + world-model rollout; early-warning threshold; onset/lead-time.	"class X likely in +N windows", infiltration curve, earliest-alarm step.
PREVENT	the response ladder keyed off verdict + forecast: protected-address allow-list -¿ rate-limit -¿ eBPF/XDP drop -¿ firewall rule -¿ quarantine VLAN -¿ host isolate; human-in-loop above rate-limit; auto-rollback; audit.	contained blast radius, recommended hardening.

Effort tags below: **[now]** works today; **[+sig]** one signature rule; **[+feat]** one feature/parser; **[+model]** a new model head or module; **[+int]** an enterprise integration (firewall / NAC / EDR / AD / WAF / DNS-RPZ).

12 Attack-by-attack

DoS (single-source: SYN/ACK/RST flood, slowloris, slow-read, connection exhaustion)

Flag **[now]** ANN `DoS` class + the `syn-flood` signature (SYN ¿= 20, ACK/SYN ¡= 0.2, ¿= 100 pkts/s); packet-rate + half-open ratio. Slowloris **[+sig]**: count of long idle half-open connections.

Predict **[now]** world-model rollout on packet-rate + SYN-ratio slope -¿ infiltration curve; earliest-alarm step at threshold 0.60.

Prevent **[+int]** rate-limit -¿ eBPF/XDP drop of the source; recommend SYN cookies and connection timeouts.

DDoS (volumetric L3/4, amplification/reflection, L7 HTTP flood, pulse-wave)

Flag **[now]** ANN `DDoS` + the `rolling-source-fanin-flood` signature; source-IP entropy. Reflection **[+sig]**: response-without-request + amplification ratio by port (53/123/11211/1900/389).

Predict **[now]** fan-in ramp + per-victim flow-ratio trend feed H1–H6; **[+int]** edge/NetFlow sensors see the ramp minutes before saturation.

Prevent **[+int]** BGP blackhole / Flowspec / upstream scrubbing; XDP drop by source prefix; WAF rate rules + JS challenge for L7.

Phishing (spearphish link/attachment, credential-harvest site, BEC)

Flag [+**feat**] DNS to newly-registered / typosquat / punycode domains; TLS SNI + certificate age/issuer for look-alike sites; [+**int**] mail-gateway logs. Post-click: [**now**] the callback flow after the payload runs is caught by the signature + ANN.

Predict [+**model**] "recon domain resolved but not yet contacted" watch-list; a fleet-wide spike in look-alike DNS = campaign early warning.

Prevent [+**int**] DNS-RPZ sinkhole the domain; block egress to the IP; force a password reset via the AD API; push the IOC to the mail gateway.

Man-in-the-middle (ARP spoof, LLMNR/NBNS poisoning, rogue DHCP, DNS spoof, SSL strip, rogue AP, BGP hijack)

Flag [+**feat**] IP-to-MAC binding churn, gratuitous-ARP rate, duplicate IP; rogue DHCP = unexpected OFFER source; DNS response anomaly / DNSSEC-fail; HTTP where HTTPS was expected (SSL strip). Rogue AP needs [+**int**] WIPS.

Predict ARP-storm precursor before poisoning completes; BGP: origin-AS change against an RPKI baseline.

Prevent [+**int**] trigger Dynamic ARP Inspection / DHCP snooping / port-security via NAC; quarantine the port/MAC to an isolation VLAN.

Port scanning / reconnaissance (SYN/FIN/NULL/Xmas/UDP scan, slow scan, distributed scan, OS/version fingerprint, vuln scan)

Flag [**now**] the `port-sweep` signature (¿= 20 distinct dst ports, mean fwd bytes ¡= 400) + the packet-order classifier; ANN `PortScan`.

Predict [**now**] *the single best leading indicator* — scan then exploit. Scan intensity + target spread feed the forecaster; the world model raises "exploit likely at host X in +N windows".

Prevent [+**int**] tarpit / rate-limit / block the scanner prefix; move the probed host behind a stricter ACL; [**track H**] redirect to a honeypot for a clean label.

Brute force / credential attacks (SSH/RDP/SMB/HTTP-auth brute force, password spraying, credential stuffing, Kerberoasting)

Flag [+**feat**] a failed-auth graph (source IP x account x service), velocity + failure ratio; [+**int**] AD / auth-log fusion for spraying (1 try, many accounts) and Kerberoasting (rare SPN TGS requests).

Predict a distributed low-and-slow login ramp across the fleet -¿ "account-takeover attempt in progress".

Prevent [+**int**] block the source, enforce MFA / lockout, disable the targeted account, impossible-travel rule.

Malware C2 / botnet (HTTP/HTTPS beaconing, DNS C2, Cobalt Strike / Sliver, fast-flux, DGA domains)

Flag [+**feat**] a beacon detector = interval + jitter clustering on periodic small egress; JA3/JA4 + malleable-profile fingerprint; DGA = query-name entropy + length + NXDOMAIN rate. [**now**] the loud ones are already caught.

Predict a new periodic egress edge emerging after compromise = "host X is now controlled"; feeds the world model -¿ forecast lateral movement next.

Prevent [+**int**] egress allow-list, DNS sinkhole the C2, XDP drop the IP, isolate the host via EDR; push the IOC fleet-wide.

Lateral movement (SMB / PsExec / WMI / WinRM / RDP, pass-the-hash, SMB relay)

Flag [+**model, track C**] the communication-graph GNN — new east-west host-pair edges, unusual service use, anomalous paths; [+**feat**] admin-share access outside the baseline.

Predict [**track C**] the first anomalous east-west edge -¿ forecast the next hops (graph link-prediction). This is the "Campaign" view.

Prevent [+**int**] micro-segmentation rule push, quarantine both endpoints, kill the SMB session, disable the credential.

Data exfiltration (bulk HTTPS upload, DNS tunnelling, ICMP tunnelling, cloud-storage exfil, low-and-slow)

Flag [+**feat**] upload/download byte asymmetry per host; DNS query volume + entropy + TXT/NULL record rate; ICMP payload size/entropy; destination novelty (never-seen ASN/domain).

Predict [+**model**] internal "staging" (large internal reads / share enumeration) precedes egress by minutes–hours -¿ warn before data leaves.

Prevent [+**int**] egress byte-cap, block the destination, DNS-RPZ, DLP / CASB trigger, isolate the host.

Ransomware (delivery -¿ C2 -¿ lateral spread -¿ share enumeration -¿ mass encryption over SMB)

Flag [+**feat**] SMB file-op rate + read-then-write-rename burst on shares; internal SMB fan-out; a workstation contacting the backup server; [**track H**] a canary-file touch = instant high confidence.

Predict [**track C+E**] the lateral phase (often hours) is the warning window — graph anomaly + forecast "encryption imminent on segment Y".

Prevent [+**int**] isolate the host (EDR), block SMB between segments, disable the spreading account, snapshot/lock backups.

Web application attacks (SQLi, XSS, RCE, LFI/RFI, path traversal, SSRF, XXE)

Flag [+**feat**] a char-level model on request strings from cleartext HTTP or [+**int**] WAF-mirrored logs; payload signatures; SSRF = the app server suddenly talks to internal / metadata IPs.

Predict a recon 404-scan / parameter-fuzz burst precedes the working exploit.

Prevent [+**int**] push a WAF block rule, block the source, rate-limit the endpoint.

DNS attacks (cache poisoning, hijack, tunnelling, DGA, NXDOMAIN flood, subdomain enumeration)

Flag	[+**feat**] a DNS parser: response anomaly / DNSSEC-fail (poisoning); query entropy + NXDOMAIN rate (DGA/tunnel); rapid unique-subdomain rate (enumeration).
Predict	a DGA burst = malware present but pre-C2 -¿ warn before the successful callback.
Prevent	[+**int**] RPZ sinkhole, force resolver DNSSEC validation, block the client.

Spoofing (IP / MAC / email)

Flag	[+**feat**] IP spoof = TTL / OS-fingerprint vs the claimed source, RFC2827 ingress violation; MAC spoof = OUI vs behaviour; email via [+**int**] SPF/DKIM/DMARC in mail logs.
Predict	spoofed-source recon usually precedes a reflection DDoS or a session attack.
Prevent	[+**int**] uRPF / BCP38 filter push, port-security, drop at the edge.

VPN / remote-access attacks (VPN brute force, appliance-CVE exploit, session hijack)

Flag	[+**feat**] auth-failure rate on the concentrator; post-auth behaviour that doesn't match the user baseline; exploit = a malformed handshake / unexpected admin path.
Predict	brute-force ramp -¿ successful login -¿ immediate internal scan = takeover in progress.
Prevent	[+**int**] block the source, force MFA re-auth, kill the tunnel, apply the CVE patch advisory.

Wireless attacks (deauth flood, evil twin, WPA handshake capture, KRACK)

Flag	[+**int**] needs a WIPS feed — deauth-frame rate, duplicate BSSID/SSID, rogue-AP MAC.
Predict	—
Prevent	[+**int**] WIPS containment, 802.1X + PMF, rogue-AP switchport shutdown.

IoT / OT attacks (Mirai-style recruitment via telnet/23, Modbus/DNP3 unauthorised write, OT protocol scan)

Flag	[+**feat**] a protocol-aware parser + a command allow-list model; a telnet/23 spike; unexpected function codes / write-coil operations.
Predict	an OT read-burst / enumeration precedes a write attack.
Prevent	[+**int**] OT-DMZ rule, unidirectional-gateway recommendation, block the source, allow-list commands.

Insider threat

Flag [+**model**] a per-user / per-host behaviour baseline — off-hours bulk transfer, access to systems outside the user's role, mass download.
Predict an escalating access-scope trend before the big exfil.
Prevent [+**int**] alert + step-up auth + revoke access; DLP hold.

Supply-chain / third-party

Flag [+**feat**] egress to a new update / CI endpoint; a build server talking to the internet unexpectedly; a vendor-VPN source doing internal recon.
Predict anomalous build-infra egress before the payload spreads.
Prevent [+**int**] egress allow-list for build infra, isolate the vendor segment, SBOM + signed-artifact check.

TLS/SSL attacks (downgrade, strip, weak cipher, certificate abuse, JA3 mismatch)

Flag [+**feat**] a TLS parser — version/cipher downgrade, a self-signed / short-lived certificate to a sensitive destination, a client JA3 that doesn't match the claimed app.
Predict —
Prevent [+**int**] block the connection, enforce a minimum-TLS policy, alert on certificate anomalies.

Cryptojacking

Flag [+**feat**] a Stratum-protocol fingerprint / known mining-pool IP+port; steady long-lived low-bandwidth egress with a mining JA3.
Predict a new host joining a pool = a compromise indicator.
Prevent [+**int**] block pool egress, isolate the host.

Living-off-the-land (certutil / bitsadmin download, PowerShell remoting, WMI exec)

Flag [+**feat**] a user-agent + destination pattern for LOLBin downloads; WinRM/5985 from non-admin hosts; correlate with [+**int**] EDR process telemetry.
Predict the first LOLBin download -¿ forecast lateral movement / C2 next.
Prevent [+**int**] block the download URL, isolate the host, disable WinRM on the segment.

Evasion / protocol abuse (fragmentation, TTL games, domain fronting, packet padding, timing evasion)

Flag [+**feat**] a fragment-overlap / tiny-fragment detector; SNI $\neq$ Host (fronting); [**B track**] a packet size/timing sequence model is padding-resistant.
Predict —
Prevent [+**int**] reassemble-then-inspect, block fronting CDNs by policy, drop overlapping fragments.

Zero-day / novel exploit

Flag	[+**model, D track**] a self-supervised anomaly model trained on benign traffic — scores "never seen this behaviour"; no signature needed.
Predict	an anomaly cluster forming across hosts = a campaign, even without a label.
Prevent	[+**int**] quarantine the anomalous segment, capture full PCAP for IR, human triage.

13 Coverage — an honest table

Coverage	Attacks
Strong today	DoS, volumetric DDoS, port scan / reconnaissance.
Partial + small add	SYN-flood variants, reflection DDoS, brute force, exfiltration (byte asymmetry), spoofing, cleartext web attacks.
Needs a new module	MITM, phishing, C2 / beaconing, lateral movement (graph), DNS attacks, the ransomware chain, TLS attacks, LOLBins, insider, IoT/OT, zero-day anomaly, wireless.
Needs integration only	firewall / NAC / EDR / AD / WAF / DNS-RPZ response actions, mail + auth-log fusion, WIPS, DLP.

Rollout order that buys the most coverage per unit of effort:

1. **Response integrations** — firewall API + NAC quarantine + EDR isolate + AD disable. Turns every existing detection into a prevention.
2. **DNS + TLS parsers** — unlocks phishing, C2, DGA, tunnelling, TLS attacks and domain fronting at once.
3. **Auth-log / AD fusion** — brute force, spraying, Kerberoasting, lateral movement, insider.
4. **Communication-graph GNN** — lateral movement, ransomware spread, APT campaigns, the "Campaign" view.
5. **Self-supervised anomaly head** — zero-day and everything without a signature.
6. **Real-time forecasting + time-to-attack + LLM triage** — turns all of the above into "X likely at host Y in N minutes" with a plain-English playbook.

Part IV — Real attacks in India

Incident responders in India follow CERT-In playbooks, not research papers. Below is what actually happened, what was done, and the research that addresses that *class* of attack.

Incident	Network view	How handled	Research / approach
Kudankulam NPP, Oct 2019	DTrack (Lazarus) on the administrative network via an infected personal PC; the "air-gap" assumption failed.	NPCIL isolated the admin net; CERT-In + IB probe; removable-media policy tightened.	removable-media control; host anomaly detection; ICS/OT network baselining.
AIIMS Delhi ransomware, Nov 2022	phishing -¿ lateral movement (weak segmentation) -¿ ∼100 servers encrypted, ∼40M records, the Hospital Information System down ∼2 weeks.	isolate hosts; CERT-In + NIC + E&Y; sanitise + AV sweep thousands of machines; phased restore; segmentation added.	ransomware early detection via file-op + SMB fan-out + beaconing signals; email-attachment NLP; immutable backups + canary files.
Cosmos Bank, Aug 2018	malware -¿ a parallel malicious ISO8583 ATM/POS switch; the CBS link cut; $13.5M via SWIFT + cloned-card cashout across 28 countries.	disconnect SWIFT; RBI directions; forensic reconstruction.	cross-entity behavioural analytics (users x frontend x backend x jump servers x SWIFT); switch + jump-server integrity monitoring.
Oil India Ltd, 2022	ransomware, ∼Rs 57 cr demand, field IT hit.	isolate, rebuild, CERT-In.	as AIIMS.
Power sector / RedEcho, 2020–21	China-linked (Recorded Future) targeting of regional load-despatch centres during the border stand-off; malware staged in grid infrastructure.	CEA audits, air-gap review, CERT-In / NCIIPC advisories.	ICS network baselining; grid-SCADA anomaly detection; NCIIPC critical-infra guidance.
Hacktivist DDoS waves, 2023–24	#OpIndia / G20 / Independence Day; L3/L4/L7 floods + defacements; 1000+ sites; government the top target category.	CERT-In sector advisories; upstream scrubbing; WAF/CDN; rate rules.	eBPF/XDP DDoS mitigation; ML DDoS classifiers on source entropy and fan-in.
ICMR, Oct 2023	∼81.5 cr Aadhaar/passport records offered for sale (data layer, not network).	CBI probe; CERT-In.	DLP + database activity monitoring + egress anomaly detection.
SpiceJet, May 2022	ransomware attempt; flight departures delayed.	contain + restore; DGCA notified.	as AIIMS.
BSNL, 2023–24	subscriber database breach, repeated.	CERT-In notice, audit.	database monitoring + credential hygiene.

Incident	Network view	How handled	Research / approach
Air India (via SITA), 2021	third-party passenger-service-system breach; ∼4.5M records.	vendor-led IR; password resets.	supply-chain: vendor egress monitoring, zero-trust to third parties.

Two more for the *forecasting* angle: the **Mumbai power outage (Oct 2020)** — a disputed cyber link that prompted the RedEcho disclosure — and **Operation Sindoor (May 2025)**, where roughly 200,000 probing and attack attempts hit grid infrastructure within days: a textbook "forecast from reconnaissance" case.

Part V — The 2026 AI-era landscape

14 Can the present model and data solve the coverage gap?

No — not on its own. The trained ANN + LSTM + world model + CICFlowMeter + CICIDS2017 is a solid *volumetric-DoS / DDoS / scan detector plus a short-horizon forecasting prototype*. The coverage gap (phishing, MITM, C2, lateral movement, exfiltration, ransomware…) is a **sensor and label problem**, not a model problem. More training epochs on the same 77 flow features and 4 classes cannot conjure DNS / TLS / SMB / auth visibility that was never captured.

What *is* achievable right now with AI and no new capture hardware:

- a **self-supervised anomaly head** (autoencoder / Isolation Forest / deep SVDD) on the existing 77 features — an "unknown attack" score with no new labels;
- **retrain the ANN** on modern NetFlow-format datasets (NF-CSE-CIC-IDS2018-v2, NF-UNSW-NB15-v2, CIC-IoT-2023) that share the feature shape — modern traffic, 10–20 attack classes, and it then runs on router NetFlow export, not just local packet capture;
- an **LLM triage layer** on top of the existing MITRE mapper;
- **keep the LSTM / H1–H6 / world-model forecaster** as the differentiator — almost no open-source project does short-horizon attack-state forecasting.

15 Direction of the field

Hand-crafted flow features are giving way to **network foundation models** (netFound, ET-BERT, NetMamba, TrafficLLM — transformers pretrained on raw packet bytes or headers) and to **self-supervised graph models** (Anomal-E / E-GraphSAGE — learn "normal" from the communication graph, score live edges by surprise).

16 Open-source projects that have already solved large parts

Project	Scale	What it already solves
Slips (Stratosphere-LinuxIPS, CTU Prague)	∼900 stars, very active	behavioural Python IDS/IPS: per-IP *profiles* split into *time windows* with dozens of features; ML models + 40+ threat-intel feeds + heuristics; ingests PCAP / Zeek / Suricata / Argus / NetFlow; detects scan, DDoS, C2, exfiltration, long connections, DGA. *The closest architectural twin — add our forecaster as a Slips module.*
Malcolm (CISA + Idaho National Lab)	∼2–3k stars	a full network traffic-analysis suite: Zeek + Suricata + Arkime + OpenSearch, dockerised, PCAP -¿ enriched dashboards, OT-aware. *Ship our detector inside it.*
Wazuh	∼11k stars	open SIEM / XDR: host + network + log correlation, MITRE-mapped rules, active-response actions. *The enterprise SIEM + response gap.*
Zeek / Suricata / Arkime	∼7k / 5k / 6k stars	the sensor layer we lack: DNS / TLS (JA3/JA4) / HTTP / SMB / SSH logs; signature IDS; large-scale full-PCAP capture + hunt.
netFound	open weights on HuggingFace	a transformer foundation model on packet headers + payload + flow; a container from raw PCAP to inference.
ET-BERT / NetMamba / TrafficLLM	∼400 stars / research	pretrained encrypted-traffic classifiers; TrafficLLM fine-tunes an LLM for multi-task traffic analysis.
Anomal-E / E-GraphSAGE	research	self-supervised GNN on the flow graph; trains on NF-CSE-CIC-IDS2018-v2 (same lineage as our data).
AI-SOC platforms (agentic-soc-platform ∼1.1k, Vigil SOC, AiSOC, zhadyz/AI_SOC)	varies	an LLM triage / investigation / response layer; zhadyz/AI_SOC is a working blueprint (Foundation-Sec-8B + Wazuh + TheHive + RAG).

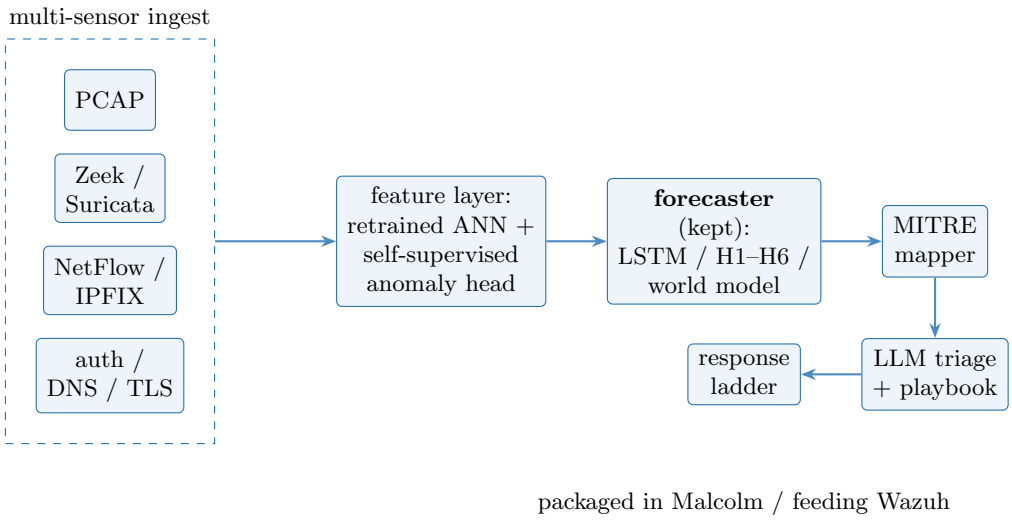


Figure 22: Target architecture. Coverage comes from adopting solved sensor/SIEM pieces; the forecasting layer stays ours.

Part VI — The roadmap

Stance: adopt the solved open-source pieces for coverage; keep the forecaster as the thing that is genuinely new. Steps in order.

17 Step 1 — Widen the front door

Add a Zeek / Suricata / NetFlow ingester beside the PCAP path (or run the whole pipeline as a **Slips module**). New `backend/ingest/` + a `/api/ingest/` route. This alone removes the "must run local packet capture on the box" limit — the system then works on a span port or a collector.

18 Step 2 — Refresh the detector

Retrain the ANN on **NF-CSE-CIC-IDS2018-v2 / NF-UNSW-NB15-v2 / CIC-IoT-2023** (NetFlow-format, labelled, same feature shape) and widen the label set past four. Add a **self-supervised anomaly head** (autoencoder / Isolation Forest / deep SVDD) on the current 77 features for unknown / zero-day — trains on benign traffic only, needs no new labels.

19 Step 3 — Track C: communication-graph GNN

Hosts are nodes, flows are edges. Train a self-supervised link-prediction model on benign traffic (E-GraphSAGE / Anomal-E style); score live edges by surprise. New `backend/graph/` + `/api/graph/` + a **"Campaign"** dashboard tab for lateral movement / botnet / APT. Trains on NF-CSE-CIC-IDS2018-v2.

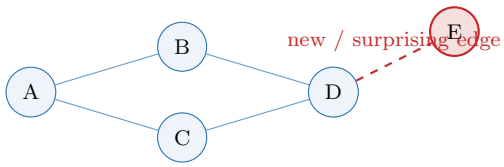


Figure 23: Track C. Almost every multi-stage attack shows up as a new or surprising edge in the host graph before it shows up in any single flow.

20 Step 4 — Tracks E + G: real-time forecasting + LLM triage

Replace "row-order proxy seconds" with real timestamps. Add a **time-to-attack** survival / hazard head ("attack within T minutes: p") and a Hawkes point-process for attack arrival, beside the world model. Add a **local security-LLM panel** (Foundation-Sec-8B style) on top of `mitre/mapper.py`: summarise an alert, rank techniques, draft a playbook, explain "why flagged", correlate threat intel. Advisory only — it never writes a rule.

21 Step 5 — Tracks H + I: deception + graduated response

Deception: a few honeypot services + honey-tokens (fake credentials, fake share). Any interaction is a near-zero-false-positive label *and* an early-warning event; it feeds steps 2–4.

Graduated response, building on the `config.json` `response.protected_addresses` allow-list already added:

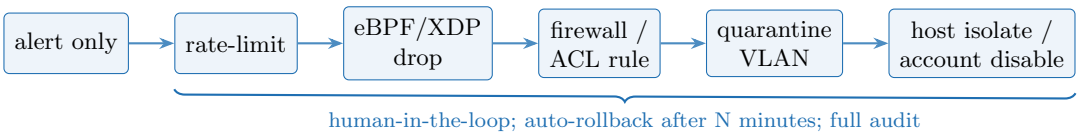


Figure 24: Track I. Escalation is gated: anything above rate-limit needs an operator, every action is logged, and everything rolls back on a timer.

22 Step 6 — Package for enterprise

Ship the detector + forecaster inside **Malcolm**; feed **Wazuh** for correlation and response; keep this dashboard as the analyst console.

23 What this is not

Steps stay honest about scope: the forecaster is short-horizon and window-based until step 4; the graph and anomaly models need the modern datasets from step 2; response actions above "alert" are operator-gated. Nothing here sends traffic off the box.

Part VII — From NIDS to XDR

24 Why PCAP alone is blind

PCAP is often called the ultimate source of truth — it records exactly what crossed the wire. Two realities bottleneck it against a modern threat matrix:

- **Ubiquitous encryption.** Ten years ago most web-app attacks (SQLi, XSS), phishing payloads and C2 beacons were cleartext, so payload inspection worked. Today over 90% of traffic is encrypted. If an attacker downloads a living-off-the-land binary or brute-forces an HTTPS login, PCAP sees only a TLS stream between two IPs — it cannot tell a Cobalt Strike payload from a PDF download.
- **Visibility scope.** A tap or span port only sees the traffic on *its* segment. Endpoint actions, identity events and wireless activity never cross it.

Visibility	Attack types	Why PCAP succeeds or fails
High (directly captured)	DoS/DDoS, port scans, DNS attacks, spoofing, IoT/OT, protocol evasion	these manipulate L3/L4 headers, flow rates, or use unencrypted protocols (DNS, Telnet, Modbus). PCAP is irrefutable ground truth.
Partial (behavioural / metadata)	malware C2, lateral movement, ransomware, exfiltration, cryptojacking	payloads are encrypted. PCAP must rely on metadata: flow volume, timing (beacon jitter), TLS handshakes (JA3/JA4/SNI), byte asymmetry.
Low (needs integrations)	phishing, brute force, web-app attacks, insider threat, LOLBins, wireless	activity is inside encrypted tunnels, on the endpoint (EDR), in auth systems (AD), or over the air (WIPS). Standard PCAP is blind.

Table 21: Attack-visibility breakdown. NIDS today lives in the "High" row plus the metadata half of "Partial".

25 The SOC Visibility Triad + XDR fusion

When PCAP is blinded, move the detection boundary to where data is either unencrypted or actively executing. Four additions:

- **Endpoint telemetry (EDR / Sysmon)** — catches ransomware, zero-days, LOLBins. Sees the action before encryption wraps it: process lineage (Word spawns a hidden PowerShell that makes an outbound connection), credential-theft attempts against LSASS, and host-level networking that ties a connection back to the exact process that made it.
- **Identity / authentication logs (AD / IAM)** — catches lateral movement, insider threats, brute force. Track the credential, not the packet: Windows Event 4624 (network logon) with 4688 (process creation) — one user account logging into 15 servers via SMB in five minutes is a red flag; Kerberoasting shows as unusual SPN ticket requests; "impossible travel" from a cloud IdP.
- **Layer-7 proxies and WAF (AppSec)** — catches web-app attacks and exfiltration. A WAF holds the TLS certificate, decrypts inbound HTTPS, inspects the raw payload for SQLi / path traversal, and blocks. A secure web gateway decrypts *outbound* traffic and inspects it for C2 beaconing, phishing domains and data exfiltration before it leaves.
- **XDR data fusion (behavioural analytics)** — no single alert is enough for a low-and-slow campaign. Baseline what a service account normally does; if it connects to a new segment (Net-

Flow), uses a rare administrative share (EDR/Sysmon) and authenticates out of hours (identity), the platform fuses those low-confidence anomalies into one high-confidence "campaign in progress" alert.

26 The architecture — six tiers

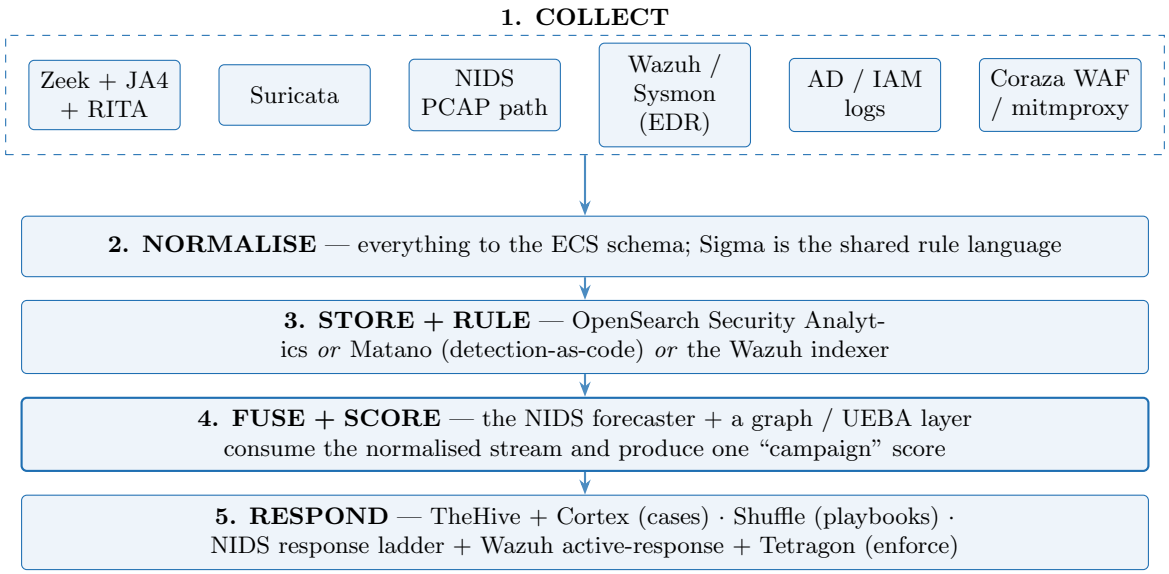


Figure 25: NIDS stays the network sensor and the forecasting brain (tier 4). The Triad adds three more collection streams; a normalise + store layer gives one schema; response is a dedicated tier.

27 What to install — open-source tools, and where each plugs into NIDS

Triad layer	Tool (GitHub)	Adds / integration point
Network metadata	`zeek/zeek` + `FoxIO-LLC/ja4` + `activecm/rita`	DNS / TLS / HTTP / SMB / SSH logs; JA4 fingerprints; per-pair beacon score 0–1. New `backend/ingest/zeek` + `/api/ingest/zeek`; feed the fields + JA4 + beacon score into the 28-number window state; a "Campaign / C2" tier beside `signatures.py`.
Signature IDS	`OISF/suricata` + ET Open rules	protocol anomalies, known-bad, JA3. Alerts into the fusion store; a signature tier alongside the existing rule layer.
Endpoint (EDR)	`wazuh/wazuh` agents + Sysmon (`SwiftOnSecurity/sysmon-config` or `olafhartong/sysmon-modular`)	process lineage, LSASS-access blocks, host netconn↔process. Correlate a NIDS network verdict with the exact process that caused it.
Host forensics	`Velocidex/velociraptor` *or* `osquery/osquery` + `fleetdm/fleet`	on-demand "what happened on this host?" hunts, triggered from a case.

Triad layer	Tool (GitHub)	Adds / integration point
Identity / auth	Windows 4624 / 4688 / 4769 via Wazuh or `elastic/beats` (Winlogbeat); `SigmaHQ/sigma`; `wagga40/Zircolite`	lateral-movement velocity, Kerberoasting SPN requests, impossible travel. Auth-velocity + lateral-graph features feed the forecaster.
L7 inbound (WAF)	`corazawaf/coraza` (Go, OWASP, active) *or* `chaitin/safeline` *or* `bunkerity/bunkerweb` + `coreruleset/coreruleset`	terminate inbound HTTPS, inspect payload for SQLi / XSS / traversal, block. WAF alerts into fusion; a block is a NIDS *prevent* action.
L7 egress (SWG / DLP)	`mitmproxy/mitmproxy` *or* Squid + SSL-bump *or* BunkerWeb as egress proxy	decrypt egress → see C2 beacons, phishing domains, data exfiltration. Decrypted egress metadata into the same window features.
Fusion / SIEM	`opensearch-project/security-analytics` *or* `matanolabs/matano` (Python detection-as-code, auto-imports Sigma) *or* the Wazuh indexer	one store, ECS schema, correlation. The fused stream is the forecaster's new input.
Cases + SOAR	`TheHive-Project/TheHive` + `TheHive-Project/Cortex` + `Shuffle/Shuffle` (or `n8n-io/n8n`)	a NIDS ATTACK verdict opens a case with the forecast + MITRE map attached; automated IOC enrichment; response playbooks.
Enforcement	Wazuh active-response; `cilium/tetragon` (in-kernel block / kill)	the top rungs of the NIDS response ladder.

28 XDR-in-a-box — if you want less assembly

- `Security-Onion-Solutions/securityonion` — Zeek + Suricata + Elastic + TheHive + Sigma + Playbook in one distribution. Closest thing to open-source XDR you can deploy in a day.
- `idaholab/Malcolm` — Zeek + Suricata + Arkime + OpenSearch, dockerised, OT-aware (CISA + Idaho National Lab). Best if you want to *embed the NIDS forecaster* rather than adopt a whole SOC.
- `wazuh/wazuh` — already an XDR + SIEM in one agent / manager / dashboard; the fastest path to endpoint + identity + file integrity + active response.

All of the above are maintained by real organisations — CNCF (Tetragon), OWASP (Coraza, Core Rule Set), OISF (Suricata), the Zeek project, FoxIO (JA4), Chaitin (SafeLine), Active Countermeasures / SsoT (RITA), Wazuh Inc., Idaho National Lab (Malcolm), StrangeBee (TheHive). Licences are Apache-2.0 / GPL / BSD.

29 The smallest first step for this repo

Add a **Zeek + JA4 + RITA sidecar** reading the same span port the PCAP path already uses. Pipe `conn.log` / `ssl.log` / `dns.log` and the RITA beacon score into the temporal window features. That one bolt-on lifts C2, exfiltration, DNS-tunnelling and TLS-attack coverage from "Low" to "Partial / High" — with *no* endpoint-agent rollout and no change to the capture path.

Appendices

Appendix A — Glossary for a non-technical reader

Flow one two-way conversation between two computers, identified by source and destination IP and port plus protocol.

Feature one number describing a flow (e.g. "bytes per second").

Softmax a model output that turns raw scores into probabilities that sum to 1.

Confidence the largest of those probabilities — how sure the model is.

Window a fixed 10-second slice of time; all flows in it are summarised into one row.

Sequence five consecutive windows used as input to predict the sixth.

Leakage when information from the test period sneaks into training, making results look better than they are; the audit hunts for it.

Macro-F1 an accuracy-like score that weights every class equally, so rare attacks count as much as common benign traffic.

False-positive rate
the share of benign traffic wrongly flagged.

Beaconing malware "phoning home" at regular intervals.

Lateral movement
an attacker hopping from the first compromised machine to others inside the network.

GNN (graph neural network)
a model that learns from a network of connections rather than a flat table.

Self-supervised trained on unlabelled "normal" data; anything unlike normal scores as anomalous.

Foundation model
a large model pretrained on huge unlabelled data, then fine-tuned for a specific job with little labelled data.

MITRE ATT&CK
a public catalogue of attacker techniques, each with an ID like T1595.

Kill chain the stages of an intrusion, from reconnaissance to impact.

Appendix B — Key numbers

Name	Value
Class names / softmax order	BENIGN, DDoS, DoS, PortScan
Flow feature count	77
State-vector size (temporal)	28
Unscoreable sentinel	INVALID_FEATURES
Write-back columns	Current_State, Signature_State, Effective_State

Name	Value
Verdict gate (DDoS / other)	own-class share ≥ 20% / 20%
Capture default duration	300 s
Capture safety timeout	600 s
Capture buffer	64 MiB
Parallel-extract chunk / threshold	20 000 / 20 000 packets
Signature DDoS fan-in / pkts-s / victim-ratio / top-source	20 / 100 / 0.5 / 0.25
Signature DoS SYN / ACK-ratio / pkts-s	20 / 0.2 / 100
Signature PortScan ports / max fwd bytes	20 / 400
Window length / sequence length	10 s / 5
Chronological split	70 / 15 / 15
One-step LSTM	LSTM(64) -¿ Dense(32) -¿ 2 x softmax(4); seed 42; max 40 epochs; batch 512
Multi-step horizons K	6 (direct)
World model K	6 default, 24 max; hidden 64; early-warning 0.60
Early-warning threshold selection	max attack-F1 s.t. FPR ≤ 5%
MITRE techniques covered	T1595, T1046, T1498(.001/.002), T1499 (ATT&CK v19.1)
Mapping confidence values	0.25 / 0.55 / 0.65 (heuristic, not calibrated)
Active LSTM model	v1-3a5264a499ed

Appendix C — Sources

India incidents: Kudankulam — VIF analysis; Gulf News. AIIMS — ORF; RingSafe; CM-Alliance timeline. Cosmos Bank — Securonix threat research; Indiaforensic. RedEcho / power — Outlook; Eventus. Hacktivist DDoS — GovInsider; SecurityLink India; CERT-In advisory CIAD-2023-0036.

Research: NetFlow context deep-learning survey (arXiv 2602.05594); FastFlow (arXiv 2504.02174); encrypted HTTPS stacked ensembles (Nature s41598-025-21261-6); encrypted malicious traffic NLP+DL (ScienceDirect S1389128624004304); E-GraphSAGE (arXiv 2103.16329); GNN-IDS survey (Computers & Security 2024, 10.1016/j.cose.2024.103821); graph foundation model for lateral movement (arXiv 2504.13527); heterogeneous-graph cyber anomaly survey (arXiv 2510.26307); ML cybercrime prediction review (arXiv 2304.04819); forecasting cyber threats + mitigation (ScienceDirect S0040162524006346); forecasting from incomplete data (arXiv 2004.04597); eBPF/XDP DDoS mitigation (arXiv 2508.00851); eBPF DoS detection in Kubernetes (MDPI Applied Sciences 13/8/4700); netFound (arXiv 2310.17025); NetMamba (arXiv 2405.11449); Anomal-E (arXiv 2207.06819).

Open source (Parts V–VI): github.com/stratosphereips/StratosphereLinuxIPS; github.com/idaholab/Malcolm; github.com/arkime/arkime; github.com/wazuh/wazuh; github.com/zeek/zeek; github.com/OISF/suricata; github.com/linwhitehat/et-bert; github.com/wangtz19/Awesome-NTA; github.com/zhadyz/AI_SOC; github.com/scadastrangelove/awesome-ai-security-tools; Cisco Foundation-Sec-8B.

XDR / SOC Visibility Triad (Part VII): github.com/Velocidex/velociraptor; github.com/osquery/osquery; github.com/fleetdm/fleet; github.com/FoxIO-LLC/ja4 ("How to Use JA4 Network Fingerprints in Zeek", zeek.org, Jan 2026); github.com/activecm/rita (Active Countermeasures / SsoT); github.com/corazawaf/coraza; github.com/chaitin/safeline; github.com/bunkerity/bunkerweb; github.com/coreruleset/coreruleset; github.com/mitmproxy/mitmproxy; github.com/SigmaHQ/sigma; github.com/wagga40/Zircolite; github.com/opensearch-project/security-analytics; github.com/matanolabs/matano; github.com/TheHive-Project/TheHive; github.com/TheHive-Project/Cortex; github.com/Shuffle/Shuffle; github.com/cilium/tetragon; github.com/Security-Onion-Solutions/securityonion; github.com/SwiftOnSecurity/sysmon-config; github.com/olafhartong/sysmon-modular. Reviews: packetlabs.net open-source EDR; pistack.xyz Wazuh vs Velociraptor vs osquery 2026; manageengine / exabeam open-source SIEM 2026; wafplanet.com Coraza / SafeLine / BunkerWeb 2026;

blackhillsinfosec.com “Detecting Malware Beacons with Zeek and RITA”.